HANDOUTS | MATERIAL
HIGHLIGHTED AND PROVIDED BY :

VU TALK

CLICK TO JOIN OUR WHATSAPP GROUP :

VU TALK WHATSAPP GROUP

VU TALK YouTube CHANNEL : click me

For Any Help Contact Me On Whatsapp : click me

بسم الله الرحمن الرحيم

CS201 MCQS

LECTURE 19;

1. **What is the primary characteristic of Sequential Access Files?**
   - A) They can be accessed randomly.
   - B) They are simple character files.
   - C) They are always stored in binary format.
   - D) They are stored in chunks or clusters.

   Answer: B) They are simple character files.

2. **Which function is used to open a file for reading?**
   - A) open("filename", ios::app)
   - B) tellg()
   - C) open()
   - D) seekp()

   Answer: C) open()

3. **Which mode is used to open a file in append mode, ensuring previous content is not deleted?**
   - A) ios::app
   - B) ios::out
   - C) ios::trunc
   - D) ios::in

   Answer: A) ios::app

4. **What does tellg() function do?**
   - A) Opens a file for writing.
   - B) Returns the current position for writing.
   - C) Returns the current get position of the file pointer.
   - D) Deletes the contents of a file.

   Answer: C) Returns the current get position of the file pointer.

5. Which function is used to determine the length of a file?
    - A) strlen()
    - B) tellg()
    - C) tellp()
    - D) seekg()

    Answer: B) tellg()

6. What is the purpose of seekg() function?
    - A) To determine the length of a file.
    - B) To read a file character by character.
    - C) To move the file pointer to a specified position.
    - D) To write data into a file.

    Answer: C) To move the file pointer to a specified position.

7. Which operation can only be performed from the beginning of a file?
    - A) Moving forward.
    - B) Moving backward.
    - C) Reading characters.
    - D) Deleting characters.

    Answer: A) Moving forward.

8. What does the sizeof() function do?
    - A) Returns the size of a file.
    - B) Returns the size of an array.
    - C) Returns the size of a variable or type.
    - D) Returns the current file position.

    Answer: C) Returns the size of a variable or type.

9. Which mode is necessary to open a file for both reading and writing using fstream?
    - A) ios::app
    - B) ios::in || ios::out
    - C) ios::out
    - D) ios::trunc

    Answer: B) ios::in || ios::out

10. What is the primary advantage of using write() function over character-by-character writing?
    - A) It is easier to implement.
    - B) It supports sequential access.
    - C) It is more efficient for large data.

- D) It works with binary files only.

Answer: C) It is more efficient for large data.

11. **Which function reads characters from a file in binary mode?**
    - A) `read()`
    - B) `getline()`
    - C) `putc()`
    - D) `getc()`

    Answer: A) `read()`

12. **What happens if the position specified by `seekg()` is beyond the end of the file?**
    - A) It throws an exception.
    - B) It moves the file pointer to the end of the file.
    - C) It moves the file pointer to the beginning of the file.
    - D) It has undefined behavior.

    Answer: B) It moves the file pointer to the end of the file.

13. **Which is the correct way to open a file in output mode and ensure previous contents are deleted?**
    - A) `open("filename", ios::in)`
    - B) `open("filename", ios::trunc)`
    - C) `open("filename", ios::app)`
    - D) `open("filename", ios::out)`

    Answer: B) `open("filename", ios::trunc)`

14. **What is the role of a key in a file structure?**
    - A) To access the file randomly.
    - B) To compress the file size.
    - C) To locate records within the file.
    - D) To delete specific data.

    Answer: C) To locate records within the file.

15. **In which scenario might the file size be larger than the actual data it contains?**
    - A) When using `seekp()` function.
    - B) When reading from a file sequentially.
    - C) When writing to a file in binary mode.
    - D) When writing to disk in blocks or clusters.

    Answer: D) When writing to disk in blocks or clusters.

16. **What does the `seekp()` function do?**

- A) Moves the file pointer to a specified position.
- B) Opens a file for writing.
- C) Determines the length of a file.
- D) Reads characters from a file.

Answer: A) Moves the file pointer to a specified position.

17. **How can you ensure a file is opened for both reading and writing using `fstream`?**
    - A) Use `ios::in` mode only.
    - B) Use `ios::out` mode only.
    - C) Use `ios::in || ios::out` mode.
    - D) Use `ios::app` mode.

    Answer: C) Use `ios::in || ios::out` mode.

18. **What is the benefit of using `ios::app` mode when opening a file for writing?**
    - A) It appends data without overwriting existing content.
    - B) It truncates the file to zero length.
    - C) It writes characters in binary format.
    - D) It opens the file in read-only mode.

    Answer: A) It appends data without overwriting existing content.

19. **Which function returns the current write position in a file?**
    - A) `tellg()`
    - B) `tellp()`
    - C) `seekg()`
    - D) `seekp()`

    Answer: B) `tellp()`

20. **What is the role of `seekg()` and `tellg()` functions in file handling?**
    - A) Opening and closing files.
    - B) Reading and writing characters.
    - C) Moving the file pointer and determining its position.
    - D) Appending data to the end of a file.

    Answer: C) Moving the file pointer and determining its position.

21. **Which function can be used to move the file pointer backward from the current position?**
    - A) `seekg()`
    - B) `seekp()`
    - C) `seekg(-10L, ios::cur)`
    - D) `seekp(10L, ios::beg)`

Answer: C) seekg(-10L, ios::cur)

22. What does tellp() function return?
    - A) The current read position.
    - B) The size of the file.
    - C) The current write position.
    - D) The size of a variable.

    Answer: C) The current write position.

23. Why is it important to maintain a constant record length in a file?
    - A) To compress the file size.
    - B) To prevent data corruption during updates.
    - C) To reduce the time taken for file operations.
    - D) To ensure sequential access to the file.

    Answer: B) To prevent data corruption during updates.

24. What is the disadvantage of using the merge method to insert data in the middle of a file?
    - A) It is complex to implement.
    - B) It requires sorting the file.
    - C) It can be inefficient for large files.
    - D) It requires converting the file to binary format.

    Answer: C) It can be inefficient for large files.

25. What is the function of a key in a database file?
    - A) To encrypt data.
    - B) To sort data.
    - C) To locate specific records.
    - D) To delete records.

    Answer: C) To locate specific records.

26. What does the sizeof() function return?
    - A) The size of a file.
    - B) The size of an array.
    - C) The size of a type or variable.
    - D) The current file position.

    Answer: C) The size of a type or variable.

27. Why are disks divided into logical blocks?
    - A) To reduce the file size.
    - B) To ensure file security.

- C) To optimize read and write operations.
- D) To encrypt file contents.

Answer: C) To optimize read and write operations.

28. **What does ios::in || ios::out flag indicate when opening a file with fstream?**
    - A) Read and write operations simultaneously.
    - B) Append mode.
    - C) Truncate the file.
    - D) Binary mode.

    Answer: A) Read and write operations simultaneously.

29. **What is the primary advantage of using write() function in file handling?**
    - A) It supports binary mode.
    - B) It appends data to a file.
    - C) It writes large chunks of data efficiently.
    - D) It moves the file pointer.

    Answer: C) It writes large chunks of data efficiently.

30. **When is ios::binary mode used in file handling?**
    - A) When writing character data.
    - B) When opening a file for input.
    - C) When handling large files.
    - D) When dealing with binary data.

    Answer: D) When dealing with binary data.

LECTURE 20:

What is a structure in C and C++?

- A) A collection of functions under a single name.
- B) A collection of variables under a single name.
- C) A data type that handles complex numbers.
- D) A keyword for defining arrays.

Answer: B) A collection of variables under a single name.

How are structures defined in C/C++?

- A) Using the class keyword.
- B) Using the struct keyword.

- C) Using the typedef keyword.
- D) Using the define keyword.

Answer: B) Using the struct keyword.

Which operator is used to access members of a structure?

- A) Dot (.) operator.
- B) Arrow (->) operator.
- C) Asterisk (*) operator.
- D) Bracket ([]) operator.

Answer: A) Dot (.) operator.

What can be a member of a structure?

- A) Only integers.
- B) Only characters.
- C) Variables of different types.
- D) Only arrays.

Answer: C) Variables of different types.

How are pointers to structures declared?

- A) pointer structName;
- B) ptr: structName;
- C) structName *ptr;
- D) ptr.structName();

Answer: C) structName *ptr;

Which statement initializes a structure at declaration?

- A) struct s = {1, "John"};
- B) struct s = {1, 'J'};
- C) s = {1, "John"};
- D) s = {1, 'J'};

Answer: A) struct s = {1, "John"};

How is an array of structures declared?

- A) struct arr[10];
- B) arr = {struct[10]};
- C) struct arr[10];

- D) arr[10] = struct;

Answer: C) struct arr[10];

 What is the function of the sizeof operator with structures?

- A) Calculates the size of each member of the structure.
- B) Determines the total number of elements in the structure.
- C) Computes the total size of the structure in memory.
- D) Measures the length of structure variables.

Answer: C) Computes the total size of the structure in memory.

 Which operator is used to access members of a structure through a pointer?

- A) Dot (.) operator.
- B) Arrow (->) operator.
- C) Asterisk (*) operator.
- D) Bracket ([]) operator.

Answer: B) Arrow (->) operator.

 Can structures be nested within each other?

- A) No, structures cannot contain other structures.
- B) Yes, structures can only contain arrays.
- C) Yes, structures can contain other structures.
- D) No, structures can only contain simple data types.

Answer: C) Yes, structures can contain other structures.

 How are structures passed to functions by default?

- A) By value.
- B) By reference.
- C) By pointer.
- D) By copy.

Answer: A) By value.

 Which keyword is used to declare a structure variable?

- A) class
- B) typedef
- C) struct
- D) variable

Answer: C) struct

 What does the following declaration define?

```
cpp
Copy code
struct Point {
   int x;
   int y;
} p1, p2;
```

- A) Two points named p1 and p2.
- B) A structure named Point.
- C) A function named Point.
- D) An array of points.

Answer: A) Two points named p1 and p2.

 Which operator is used to access elements of an array of structures?

- A) Dot (.) operator.
- B) Arrow (->) operator.
- C) Asterisk (*) operator.
- D) Bracket ([]) operator.

Answer: D) Bracket ([]) operator.

 How does assignment work with structures?

- A) Structures cannot be assigned.
- B) Structures are assigned member-wise.
- C) Structures are assigned by value only.
- D) Assignment is not supported with structures.

Answer: B) Structures are assigned member-wise.

 Which keyword defines a pointer to a structure?

- A) struct
- B) ptr
- C) typedef
- D) *

Answer: D) *

 What is a limitation of using structures in C/C++?

- A) Inability to pass structures to functions.
- B) Limited to storing only primitive data types.
- C) Structures cannot contain arrays.
- D) Operators like + are not defined for structures.

Answer: D) Operators like + are not defined for structures.

 When passing a structure to a function, which method preserves the original structure data?

- A) Pass by value.
- B) Pass by reference.
- C) Pass by copy.
- D) Pass by pointer.

Answer: B) Pass by reference.

 What does the `sizeof(student)` return?

- A) Number of students.
- B) Total number of elements in a structure.
- C) Total memory occupied by the structure.
- D) Size of each member of the structure.

Answer: C) Total memory occupied by the structure.

 What is the difference between `.` and `->` operators in structure handling?

- A) `.` is used for simple variables, `->` for arrays.
- B) `.` is used for pointers, `->` for non-pointers.
- C) `.` is for accessing members, `->` is for accessing array elements.
- D) `.` is for direct access, `->` is for indirect access through pointers.

Answer: D) `.` is for direct access, `->` is for indirect access through pointers.

 Which statement correctly declares an array of 20 students?

- A) `student s[20];`
- B) `students[20];`
- C) `student s = [20];`
- D) `students = student[20];`

Answer: A) `student s[20];`

 How can you initialize a structure at declaration?

- A) `struct s = {1, "John"};`
- B) `s = {1, 'J'};`
- C) `s = {1, "John"};`
- D) `s = {1, 'J'};`

Answer: A) `struct s = {1, "John"};`

 Which operator is used to access members of a structure through a pointer?

- A) Dot (.) operator.
- B) Arrow (->) operator.
- C) Asterisk (*) operator.
- D) Bracket ([]) operator.

Answer: B) Arrow (->) operator.

 Can structures contain other structures as members?

- A) No, structures cannot be nested.
- B) Yes, structures can only contain arrays.
- C) Yes, structures can contain other structures.
- D) No, structures can only contain simple data types.

Answer: C) Yes, structures can contain other structures.

 What is the primary use of `sizeof` with structures?

- A) To measure the number of elements in a structure.
- B) To determine the memory size of a structure.
- C) To calculate the average size of structure members.
- D) To find the maximum size of a structure member.

Answer: B) To determine the memory size of a structure.

 How are structures passed to functions by default?

- A) By value.
- B) By reference.
- C) By pointer.
- D) By copy.

Answer: A) By value.

 Which operator is used to access elements of an array of structures?

- A) Dot (.) operator.

- B) Arrow (->) operator.
- C) Asterisk (*) operator.
- D) Bracket ([]) operator.

Answer: D) Bracket ([]) operator.

How does assignment work with structures?

- A) Structures cannot be assigned.
- B) Structures are assigned member-wise.
- C) Structures are assigned by value only.
- D) Assignment is not supported with structures.

Answer: B) Structures are assigned member-wise.

Which keyword defines a pointer to a structure?

- A) struct
- B) ptr
- C) typedef
- D) *

Answer: D) *

What is a limitation of using structures in C/C++?

- A) Inability to pass structures to functions.
- B) Limited to storing only primitive data types.
- C) Structures cannot contain arrays.
- D) Operators like + are not defined for structures.

Answer: D) Operators like + are not defined for structures

LECTURE 21:

What is a bit?

- A) A small byte
- B) Smallest unit of memory
- C) A part of a byte
- D) A unit of a word

Answer: B) Smallest unit of memory

 How many bits make up a byte?

- A) 4
- B) 16
- C) 8
- D) 32

Answer: C) 8

 Which operator is used for bitwise AND in C/C++?

- A) &&
- B) ||
- C) &
- D) |

Answer: C) &

 What is the result of 5 & 3 in binary (base 2)?

- A) 7
- B) 1
- C) 5
- D) 3

Answer: B) 1

 What does the bitwise OR operator (|) do?

- A) Sets bits to 0 where either operand has a bit set to 0.
- B) Sets bits to 1 where either operand has a bit set to 1.
- C) Sets bits to 1 only where both operands have a bit set to 1.
- D) Inverts the bits in the operand.

Answer: B) Sets bits to 1 where either operand has a bit set to 1.

 What does the bitwise XOR operator (^) do?

- A) Sets bits to 1 where either operand has a bit set to 1.
- B) Sets bits to 0 where either operand has a bit set to 0.
- C) Inverts the bits in the operand.
- D) Sets bits to 1 only where both operands have a bit set to 1.

Answer: D) Sets bits to 1 only where both operands have a bit set to 1.

 What is the result of 12 ^ 9 in binary (base 2)?

- A) 5
- B) 3
- C) 15
- D) 7

Answer: A) 5

 What is the use of the bitwise NOT operator (~)?

- A) Performs logical negation.
- B) Inverts the bits of the operand.
- C) Performs bitwise addition.
- D) Performs bitwise subtraction.

Answer: B) Inverts the bits of the operand.

 How is a specific bit checked in a number using the bitwise AND operator?

- A) By ANDing with a number where only that bit is 1.
- B) By ANDing with a number where all bits except that bit are 0.
- C) By ANDing with a number where all bits are 1 except that bit.
- D) By ANDing with a number where that bit is 0.

Answer: A) By ANDing with a number where only that bit is 1.

 Which of the following operators is used for left shifting bits?

- A) <<
- B) >>
- C) ^
- D) ~

Answer: A) <<

 What does the expression x << 1 do to x in binary (base 2)?

- A) Shifts x left by 1 position.
- B) Shifts x right by 1 position.
- C) Performs a bitwise OR with 1.
- D) Performs a bitwise AND with 1.

Answer: A) Shifts x left by 1 position.

 Which operator is used to shift bits right in C/C++?

- A) `<<`
- B) `>>`
- C) `^`
- D) `~`

Answer: B) `>>`

 What does the expression `x >> 2` do to `x` in binary (base 2)?

- A) Shifts `x` left by 2 positions.
- B) Shifts `x` right by 2 positions.
- C) Performs a bitwise XOR with 2.
- D) Performs a bitwise OR with 2.

Answer: B) Shifts `x` right by 2 positions.

 Which of the following is true about bit manipulation with unsigned integers?

- A) They use a sign bit for negative numbers.
- B) They are limited to 16 bits.
- C) They are treated as positive numbers only.
- D) They use the most significant bit for storage.

Answer: C) They are treated as positive numbers only.

 In bit manipulation, what is the significance of the most significant bit in signed integers?

- A) It determines the data type of the integer.
- B) It is used for error checking.
- C) It indicates whether the number is positive or negative.
- D) It is always set to 1.

Answer: C) It indicates whether the number is positive or negative.

 Which bitwise operator is used to set a specific bit in a number?

- A) `&`
- B) `|`
- C) `^`
- D) `~`

Answer: B) `|`

 How can you clear a specific bit in a number?

- A) Using `x & 0`
- B) Using `x & 1`
- C) Using `x | 0`
- D) Using `x | 1`

Answer: A) Using `x & 0`

 What is the result of `x | 0`?

- A) `x`
- B) `0`
- C) `1`
- D) `~x`

Answer: A) `x`

 Which bitwise operation is reversible (i.e., can be undone)?

- A) `&`
- B) `|`
- C) `^`
- D) `~`

Answer: C) `^`

 What is the result of `x ^ x` where `x` is any integer?

- A) `0`
- B) `1`
- C) `-1`
- D) `x`

Answer: A) `0`

 How can you swap two variables `a` and `b` without using a temporary variable?

- A) `a = a ^ b; b = b ^ a; a = a ^ b;`
- B) `a = a | b; b = b | a; a = a | b;`
- C) `a = a & b; b = b & a; a = a & b;`
- D) `a = a << b; b = b << a; a = a << b;`

Answer: A) `a = a ^ b; b = b ^ a; a = a ^ b;`

 What is the purpose of bit manipulation in operating systems?

- A) To perform arithmetic calculations faster.

- B) To use less memory for data storage.
- C) To control hardware devices.
- D) To work with file permissions and attributes.

Answer: D) To work with file permissions and attributes.

 Which bitwise operator can be used to toggle a specific bit in a number?

- A) `&`
- B) `|`
- C) `^`
- D) `~`

Answer: C) `^`

 In the context of bit manipulation, what does 'bit setting' mean?

- A) Changing the value of a bit to 1.
- B) Changing the value of a bit to 0.
- C) Checking if a bit is 1.
- D) Flipping the bits in a number.

Answer: A) Changing the value of a bit to 1.

 Which bitwise operator can be used to check if a specific bit is set in a number?

- A) `&`
- B) `|`
- C) `^`
- D) `~`

Answer: A) `&`

 What is the result of `x & 1` if `x` has its least significant bit set to 1?

- A) `1`
- B) `0`
- C) `x`
- D) `-1`

Answer: A) `1`

 What is the significance of bit manipulation in data encryption?

- A) It speeds up data transfer.
- B) It compresses data.

- C) It ensures data integrity.
- D) It secures data by transforming it.

Answer: D) It secures data by transforming it.

 Which bitwise operation can be used to perform quick multiplication and division by powers of 2?

- A) &
- B) |
- C) <<
- D) >>

Answer: C) <<

 Which bitwise operator can be used to clear specific bits in a number?

- A) &
- B) |
- C) ^
- D) ~

Answer: A) &

 In the context of bit manipulation, what does 'bit masking' refer to?

- A) Manipulating all bits in a number.
- B) Using bitwise AND to check specific bits.
- C) Using bitwise OR to set specific bits.
- D) Using bitwise XOR to toggle bits.

Answer: B) Using bitwise AND to check specific bits.

Lectur#22:

 Which operator is used for bitwise NOT?

- A) &
- B) |
- C) ^
- D) ~

Answer: D) ~

 Which bitwise operator is used to set a bit at a specific position?

- A) &
- B) |
- C) <<
- D) >>

Answer: B) |

 What will be the result of 5 & 3?

- A) 7
- B) 1
- C) 2
- D) 0

Answer: B) 1

 What does the expression x << 3 do?

- A) Shifts x right by 3 bits
- B) Shifts x left by 3 bits
- C) Performs bitwise AND with 3
- D) Performs bitwise OR with 3

Answer: B) Shifts x left by 3 bits

 Which operator is used for bitwise XOR in C/C++?

- A) ^
- B) |
- C) &
- D) ~

Answer: A) ^

 What will be the output of 12 ^ 9 in bitwise XOR?

- A) 21
- B) 15
- C) 3
- D) 5

Answer: C) 3

 Which bitwise operator can be used to toggle specific bits?

- A) `&`
- B) `|`
- C) `^`
- D) `~`

Answer: C) `^`

 What is the result of `x &= 0xFF` in C/C++?

- A) Sets all bits of `x` to 1
- B) Sets all bits of `x` to 0
- C) Clears all but the lower 8 bits of `x`
- D) Inverts all bits of `x`

Answer: C) Clears all but the lower 8 bits of `x`

 Which operator is used for arithmetic right shift in C/C++?

- A) `<<`
- B) `>>`
- C) `|`
- D) `&`

Answer: B) `>>`

 In C/C++, what is the compound assignment operator for bitwise AND?

- A) `&=`
- B) `|=`
- C) `^=`
- D) `~=`

Answer: A) `&=`

 When should braces `{}` be used with a single statement in an `if` statement block in C/C++?

- A) Always
- B) Never
- C) Only if the statement is complex
- D) Optional

Answer: D) Optional

 Which of the following is true about arrays in C/C++?

- A) Arrays can only store integers
- B) Arrays have a fixed size once declared
- C) Arrays can be resized dynamically
- D) Arrays can only have one dimension

Answer: B) Arrays have a fixed size once declared

 What does the break statement do in C/C++?

- A) Exits the program
- B) Exits the loop or switch statement
- C) Skips the current iteration of the loop
- D) Jumps to a specific label

Answer: B) Exits the loop or switch statement

 Which loop in C/C++ executes its body at least once?

- A) for loop
- B) while loop
- C) do-while loop
- D) if-else statement

Answer: C) do-while loop

 In C/C++, which statement is used to transfer control to the next iteration of the loop?

- A) return
- B) continue
- C) break
- D) exit

Answer: B) continue

 What is a pointer in C/C++?

- A) A variable that stores the address of another variable
- B) A variable that stores the value of another variable
- C) A variable that stores a character
- D) A variable that stores an integer

Answer: A) A variable that stores the address of another variable

 How are pointers and arrays related in C/C++?

- A) Pointers can only point to arrays
- B) Arrays can only contain pointers
- C) An array's name is a pointer to its first element
- D) Pointers cannot be used with arrays

Answer: C) An array's name is a pointer to its first element

 What is the purpose of using `malloc()` function in C/C++?

- A) To allocate memory for variables
- B) To deallocate memory for variables
- C) To declare a pointer
- D) To declare an array

Answer: A) To allocate memory for variables

 In C/C++, how do you declare a pointer to an integer variable `num`?

- A) `int *num;`
- B) `int num*;`
- C) `int #`
- D) `int *num();`

Answer: A) `int *num;`

 What is the output of `printf("%d", sizeof(int))` in C/C++?

- A) 2
- B) 4
- C) 6
- D) Depends on the compiler

Answer: B) 4

 Which header file is required to use `malloc()` and `free()` functions in C/C++?

- A) `<stdlib.h>`
- B) `<stdio.h>`
- C) `<math.h>`
- D) `<ctype.h>`

Answer: A) `<stdlib.h>`

 What is the purpose of the `sizeof` operator in C/C++?

- A) Calculates the size of a variable

- B) Calculates the size of an array
- C) Returns the number of elements in an array
- D) Calculates the sum of array elements

Answer: A) Calculates the size of a variable

 Which of the following is true about void pointers in C/C++?

- A) They cannot be dereferenced
- B) They can only point to integers
- C) They cannot be passed to functions
- D) They have a fixed size

Answer: A) They cannot be dereferenced

 What is the purpose of the strcpy() function in C/C++?

- A) Concatenates two strings
- B) Compares two strings
- C) Copies a string to another string
- D) Converts a string to uppercase

Answer: C) Copies a string to another string

 Which loop in C/C++ is best used when the number of iterations is known beforehand?

- A) while loop
- B) do-while loop
- C) for loop
- D) switch statement

Answer: C) for loop

 What is the purpose of the getchar() function in C/C++?

- A) Reads an integer from the keyboard
- B) Reads a character from the keyboard
- C) Reads a string from a file
- D) Writes a character to the screen

Answer: B) Reads a character from the keyboard

 Which operator in C/C++ is used to access the value at the address stored in a pointer?

- A) *
- B) &
- C) %
- D) ~

Answer: A) *

 What is the purpose of the typedef keyword in C/C++?

- A) Defines a new type
- B) Declares a variable
- C) Allocates memory
- D) Assigns a value

Answer: A) Defines a new type

 Which statement is used to define constants in C/C++?

- A) let
- B) const
- C) final
- D) define

Answer: B) const

 What is the output of the following code snippet in C/C++?

c
Copy code
```
int a = 10, b = 20;
int *ptr1 = &a, *ptr2 = &b;
*ptr1 += *ptr2;
printf("%d", a);
```

- A) 10
- B) 20
- C) 30
- D) 40

Answer: C) 30

Lecture#23

1. What is the purpose of the preprocessor in C programming?

a) To compile the code

b) To enhance the code

c) To execute the code

d) To debug the code

Answer: b) To enhance the code

1. What is the #include directive used for?

a) To define macros

b) To include header files

c) To compile the code

d) To execute the code

Answer: b) To include header files

1. What is the difference between #include

a) The first includes system header files, while the second includes user-defined header files

b) The first includes user-defined header files, while the second includes system header files

c) The first is used for C code, while the second is used for C++ code

d) The first is used for C++ code, while the second is used for C code

Answer: a) The first includes system header files, while the second includes user-defined header files

1. What is the purpose of the #define directive?

a) To define macros

b) To include header files

c) To compile the code

d) To execute the code

Answer: a) To define macros

1. What is the benefit of using #define for constants?

a) It makes the code more readable

b) It makes the code more efficient

c) It allows for easy modification of constants

d) It is required for compilation

Answer: c) It allows for easy modification of constants

1. What is the purpose of the #ifdef directive?

a) To define macros

b) To include header files

c) To check if a symbol is defined

d) To execute the code

Answer: c) To check if a symbol is defined

1. What is the purpose of the #ifndef directive?

a) To define macros

b) To include header files

c) To check if a symbol is not defined

d) To execute the code

Answer: c) To check if a symbol is not defined

1. What is the purpose of the #if directive?

a) To define macros

b) To include header files

c) To check if a condition is true

d) To execute the code

Answer: c) To check if a condition is true

1. What is the purpose of the #else directive?

a) To define macros

b) To include header files

c) To specify an alternative block of code

d) To execute the code

Answer: c) To specify an alternative block of code

1. What is the purpose of the #endif directive?

a) To define macros

b) To include header files

c) To end a conditional compilation block

d) To execute the code

Answer: c) To end a conditional compilation block

1. What is a macro in C programming?

a) A symbolic name for a constant

b) A block of code that can be called multiple times

c) A header file

d) A library function

Answer: b) A block of code that can be called multiple times

1. What is the purpose of the #define directive for macros?

a) To define a symbolic name for a constant

b) To define a block of code that can be called multiple times

c) To include a header file

d) To execute a block of code

Answer: b) To define a block of code that can be called multiple times

1. What is the benefit of using macros?

a) It makes the code more readable

b) It makes the code more efficient

c) It allows for easy modification of code

d) It is required for compilation

Answer: c) It allows for easy modification of code

1. What is the problem with the macro definition #define square(x) x * x?

a) It does not work for negative numbers

b) It does not work for positive numbers

c) It does not follow the order of operations

d) It is not a valid macro definition

Answer: c) It does not follow the order of operations

1. How can the problem with the macro definition #define square(x) x * x be overcome?

a) By using parentheses in the macro definition

b) By using the #ifdef directive

c) By using the #ifndef directive

d) By using the #if directive

Answer: a) By using parentheses in the macro definition

1. What is the correct macro definition for the square of a number?

a) #define square(x) x * x

b) #define square(x) (x) * (x)

c) #define square(x) (x * x)

d) #define square(x) x * (x)

Answer: b) #define square(x) (x) * (x)

Lecture#24

1. What is the purpose of dynamic memory allocation?

a) To allocate memory at compile time

b) To allocate memory at runtime

c) To deallocate memory at runtime

d) To deallocate memory at compile time

Answer: b) To allocate memory at runtime

1. What is the difference between static and dynamic memory allocation?

a) Static allocation is at runtime, dynamic allocation is at compile time

b) Static allocation is at compile time, dynamic allocation is at runtime

c) Static allocation is for integers, dynamic allocation is for characters

d) Static allocation is for characters, dynamic allocation is for integers

Answer: b) Static allocation is at compile time, dynamic allocation is at runtime

1. What is the function of the malloc() function?

a) To allocate memory at compile time

b) To allocate memory at runtime

c) To deallocate memory at runtime

d) To deallocate memory at compile time

Answer: b) To allocate memory at runtime

1. What is the function of the calloc() function?

a) To allocate memory at compile time

b) To allocate memory at runtime and initialize it to zero

c) To deallocate memory at runtime

d) To deallocate memory at compile time

Answer: b) To allocate memory at runtime and initialize it to zero

1. What is the function of the free() function?

a) To allocate memory at runtime

b) To deallocate memory at runtime

c) To allocate memory at compile time

d) To deallocate memory at compile time

Answer: b) To deallocate memory at runtime

1. What is a memory leak?

a) When memory is allocated but not freed

b) When memory is freed but not allocated

c) When memory is allocated and freed at the same time

d) When memory is not allocated or freed

Answer: a) When memory is allocated but not freed

1. What is a dangling pointer?

a) A pointer that points to a valid memory location

b) A pointer that points to an invalid memory location

c) A pointer that points to a memory location that has been freed

d) A pointer that points to a memory location that has not been allocated

Answer: c) A pointer that points to a memory location that has been freed

1. What is the purpose of the realloc() function?

a) To allocate memory at runtime

b) To deallocate memory at runtime

c) To resize a previously allocated memory block

d) To initialize memory to zero

Answer: c) To resize a previously allocated memory block

1. What is the difference between malloc() and calloc()?

a) Malloc() allocates memory at runtime, calloc() allocates memory at compile time

b) Malloc() initializes memory to zero, calloc() does not initialize memory

c) Malloc() allocates memory for a single element, calloc() allocates memory for an array

d) Malloc() is used for integers, calloc() is used for characters

Answer: b) Malloc() initializes memory to zero, calloc() does not initialize memory

1. What is the purpose of the sizeof operator?

a) To get the size of a data type

b) To get the size of a variable

c) To get the size of a memory block

d) To get the size of a function

Answer: a) To get the size of a data type

1. How do you allocate memory for an array of 10 integers using malloc()?

a) malloc(10 * sizeof(int))

b) malloc(10)

c) malloc(sizeof(int))

d) malloc(10 * sizeof(char))

Answer: a) malloc(10 * sizeof(int))

1. How do you allocate memory for an array of 10 characters using calloc()?

a) calloc(10, sizeof(char))

b) calloc(10, sizeof(int))

c) calloc(10)

d) calloc(sizeof(char))

Answer: a) calloc(10, sizeof(char))

1. What is the difference between a null pointer and a dangling pointer?

a) A null pointer points to a valid memory location, a dangling pointer points to an invalid memory location

b) A null pointer points to an invalid memory location, a dangling pointer points to a valid memory location

c) A null pointer is used for integers, a dangling pointer is used for characters

d) A null pointer is used for characters, a dangling pointer is used for integers

Answer: b) A null pointer points to an invalid memory location, a dangling pointer points to a valid memory location

1. What is the purpose of the free() function?

a) To allocate memory at runtime

b) To deallocate memory at runtime

c) To initialize memory to zero

d) To resize a previously allocated memory block

Answer: b) To deallocate memory at runtime

1. What is the purpose of the #undef directive?

a) To define a macro

b) To undefine a macro

c) To include a header file

d) To execute a block of code

Answer: b) To undefine a macro

1. What is the benefit of using conditional compilation?

a) It makes the code more readable

b) It makes the code more efficient

c) It allows for easy modification of code

d) It is required for compilation

Answer: c) It allows for easy modification of code

1. What is the purpose of the #error directive?

a) To define a macro

b) To include a header file

c) To generate an error message

d) To execute a block of code

Answer: c) To generate an error message

1. What is the purpose of the #line directive?

a) To define a macro

b) To include a header file

c) To specify the line number

d) To execute a block of code

Answer: c) To specify the line number

1. What is the purpose of the #pragma directive?

a) To define a macro

b) To include a header file

c) To provide additional information to the compiler

d) To execute a block of code

Answer: c) To provide additional information to the compiler

1. What is the purpose of the #assert directive?

a) To define a macro

b) To include a header file

c) To test a condition at runtime

d) To execute a block of code

Answer: c) To test a condition at runtime

1. What is a header file in C programming?

a) A file that contains macros and functions

b) A file that contains the main function

c) A file that contains the compiler directives

d) A file that contains the library functions

Answer: a) A file that contains macros and functions

1. What is the purpose of the #include directive?

a) To define a macro

b) To include a header file

c) To execute a block of code

d) To compile the code

Answer: b) To include a header file

1. What is the difference between #include

a) The first includes system header files, while the second includes user-defined header files

b) The first includes user-defined header files, while the second includes system header files

c) The first is used for C code, while the second is used for C++ code

d) The first is used for C++ code, while the second is used for C code

Answer: a) The first includes system header files, while the second includes user-defined header files

1. What is the benefit of using header files in C programming?

a) It makes the code more readable

b) It makes the code more efficient

c) It allows for easy modification of code

d) It is required for compilation

Answer: c) It allows for easy modification of code

Lecture#25:

Here are 30 multiple-choice questions with answers based on the provided text:

1. What is the main difference between C and C++?

a) C is a high-level language, while C++ is a low-level language.

b) C is an object-oriented language, while C++ is a structured language.

c) C is a subset of C++.

d) C++ is a subset of C.

Answer: c) C is a subset of C++.

1. What is the purpose of default function arguments in C++?

a) To reduce code redundancy

b) To increase code readability

c) To provide a default value for a function parameter

d) To override function overloading

Answer: c) To provide a default value for a function parameter

1. What is the advantage of using inline functions over macros?

a) Inline functions are faster than macros

b) Inline functions are more readable than macros

c) Inline functions provide type checking, while macros do not

d) Inline functions can be used with any data type, while macros are limited

Answer: c) Inline functions provide type checking, while macros do not

1. What is function overloading?

a) Using the same function name for different purposes

b) Using different function names for the same purpose

c) Using the same function name with different parameters

d) Using different function names with different parameters

Answer: c) Using the same function name with different parameters

1. How does the compiler differentiate between overloaded functions?

a) By the return type

b) By the number and type of parameters

c) By the function name

d) By the scope of the function

Answer: b) By the number and type of parameters

1. What is the purpose of the "inline" keyword in C++?

a) To declare a function as inline

b) To declare a function as static

c) To declare a function as extern

d) To declare a function as virtual

Answer: a) To declare a function as inline

1. What is the difference between a macro and an inline function?

a) A macro is a preprocessor directive, while an inline function is a compiler directive

b) A macro is a function, while an inline function is a preprocessor directive

c) A macro is faster than an inline function

d) A macro is more readable than an inline function

Answer: a) A macro is a preprocessor directive, while an inline function is a compiler directive

1. What is the advantage of using function overloading?

a) It makes the code more readable

b) It makes the code more efficient

c) It makes the code more reusable

d) It makes the code more modular

Answer: a) It makes the code more readable

1. What is the purpose of the "default" keyword in C++?

a) To declare a default value for a variable

b) To declare a default function parameter

c) To declare a default class constructor

d) To declare a default destructor

Answer: b) To declare a default function parameter

1. How many parameters can a function have in C++?

a) A maximum of 5 parameters

b) A maximum of 10 parameters

c) There is no limit to the number of parameters

d) A minimum of 2 parameters

Answer: c) There is no limit to the number of parameters

1. What is the difference between a function declaration and a function definition?

a) A function declaration is the prototype, while a function definition is the body

b) A function declaration is the body, while a function definition is the prototype

c) A function declaration is used for function overloading, while a function definition is used for function overriding

d) A function declaration is used for function overriding, while a function definition is used for function overloading

Answer: a) A function declaration is the prototype, while a function definition is the body

1. What is the purpose of the "return" statement in C++?

a) To return a value from a function

b) To exit a function

c) To declare a variable

d) To define a function

Answer: a) To return a value from a function

1. What is the difference between a local variable and a global variable?

a) A local variable is accessible only within a function, while a global variable is accessible throughout the program

b) A local variable is accessible throughout the program, while a global variable is accessible only within a function

c) A local variable is declared inside a function, while a global variable is declared outside a function

d) A local variable is declared outside a function, while a global variable is declared inside a function

Answer: a) A local variable is accessible only within a function, while a global variable is accessible throughout the program

Lecture 26:

1. What is a class in programming?

a) A built-in data type

b) A user-defined data type

c) A function

d) A variable

Answer: b) A user-defined data type

1. What are the two main parts of a class?

a) Public and private

b) Interface and implementation

c) Data members and member functions

d) All of the above

Answer: d) All of the above

1. What is the purpose of the public part of a class?

a) To hide the implementation details

b) To show the implementation details

c) To provide an interface for the class

d) To define the data members

Answer: c) To provide an interface for the class

1. What is the purpose of the private part of a class?

a) To provide an interface for the class

b) To hide the implementation details

c) To define the data members

d) To manipulate the data members

Answer: b) To hide the implementation details

1. What are member functions?

a) Functions that are not part of a class

b) Functions that are part of a class

c) Variables that are part of a class

d) Data types that are part of a class

Answer: b) Functions that are part of a class

1. What is the purpose of member functions?

a) To manipulate the data members

b) To hide the implementation details

c) To provide an interface for the class

d) To define the data members

Answer: a) To manipulate the data members

1. What is a constructor?

a) A special kind of function that has the same name as the class

b) A special kind of variable that has the same name as the class

c) A special kind of data type that has the same name as the class

d) A special kind of function that has a different name from the class

Answer: a) A special kind of function that has the same name as the class

1. What is the purpose of a constructor?

a) To initialize the data members

b) To manipulate the data members

c) To provide an interface for the class

d) To hide the implementation details

Answer: a) To initialize the data members

1. What is the difference between a structure and a class?

a) A structure has public data members, while a class has private data members

b) A structure has private data members, while a class has public data members

c) A structure has no data members, while a class has data members

d) A structure has no member functions, while a class has member functions

Answer: a) A structure has public data members, while a class has private data members

1. What is encapsulation?

a) The concept of hiding the implementation details

b) The concept of showing the implementation details

c) The concept of providing an interface for the class

d) The concept of defining the data members

Answer: a) The concept of hiding the implementation details

1. What is inheritance?

a) The concept of creating a new class from an existing class

b) The concept of creating a new class from scratch

c) The concept of modifying an existing class

d) The concept of deleting an existing class

Answer: a) The concept of creating a new class from an existing class

1. What is polymorphism?

a) The ability of an object to take multiple forms

b) The ability of an object to take only one form

c) The ability of an object to change its form

d) The ability of an object to remain in its original form

Answer: a) The ability of an object to take multiple forms

1. What is abstraction?

a) The concept of showing only the necessary information

b) The concept of showing all the information

c) The concept of hiding all the information

d) The concept of modifying the information

Answer: a) The concept of showing only the necessary information

1. What is an object?

a) An instance of a class

b) An instance of a structure

c) An instance of a function

d) An instance of a variable

Answer: a) An instance of a class

1. What is the purpose of an object?

a) To provide an interface for the class

b) To hide the implementation details

c) To manipulate the data members

d) To represent a real-world entity

Answer: d) To represent a real-world entity

Lecture#27:

1. What is the purpose of a constructor in C++?

a) To initialize objects

b) To destroy objects

c) To manipulate data

d) To overload functions

Answer: a) To initialize objects

1. What is the name of the constructor that takes no arguments?

a) Default constructor

b) Parameterized constructor

c) Simple constructor

d) Compiler-generated constructor

Answer: a) Default constructor

1. What is the purpose of a destructor in C++?

a) To initialize objects

b) To destroy objects

c) To manipulate data

d) To overload functions

Answer: b) To destroy objects

1. When is a destructor called?

a) When an object is created

b) When an object is destroyed

c) When an object is manipulated

d) When an object is overloaded

Answer: b) When an object is destroyed

1. What is the difference between a constructor and a destructor?

a) A constructor initializes objects, while a destructor destroys objects

b) A constructor destroys objects, while a destructor initializes objects

c) A constructor manipulates data, while a destructor overloads functions

d) A constructor overloads functions, while a destructor manipulates data

Answer: a) A constructor initializes objects, while a destructor destroys objects

1. What is the purpose of utility functions in a class?

a) To manipulate data

b) To initialize objects

c) To destroy objects

d) To provide additional functionality to member functions

Answer: d) To provide additional functionality to member functions

1. What is the difference between a member function and a utility function?

a) A member function manipulates data, while a utility function provides additional functionality

b) A member function provides additional functionality, while a utility function manipulates data

c) A member function is public, while a utility function is private

d) A member function is private, while a utility function is public

Answer: a) A member function manipulates data, while a utility function provides additional functionality

1. What is the purpose of a class in C++?

a) To define a user-defined data type

b) To define a function

c) To define a variable

d) To define a loop

Answer: a) To define a user-defined data type

1. What is the difference between a class and an object?

a) A class is a user-defined data type, while an object is an instance of a class

b) A class is an instance of a user-defined data type, while an object is a user-defined data type

c) A class is a function, while an object is a variable

d) A class is a variable, while an object is a function

Answer: a) A class is a user-defined data type, while an object is an instance of a class

1. What is the purpose of encapsulation in C++?

a) To hide data and functions from outside the class

b) To show data and functions to outside the class

c) To manipulate data and functions

d) To overload data and functions

Answer: a) To hide data and functions from outside the class

1. What is the difference between public and private members of a class?

a) Public members are accessible from outside the class, while private members are not

b) Private members are accessible from outside the class, while public members are not

c) Public members are used for encapsulation, while private members are used for inheritance

d) Private members are used for encapsulation, while public members are used for inheritance

Answer: a) Public members are accessible from outside the class, while private members are not

1. What is the purpose of inheritance in C++?

a) To create a new class from an existing class

b) To create a new function from an existing function

c) To create a new variable from an existing variable

d) To create a new loop from an existing loop

Answer: a) To create a new class from an existing class

1. What is the difference between a base class and a derived class?

a) A base class is a new class, while a derived class is an existing class

b) A base class is an existing class, while a derived class is a new class

c) A base class is used for encapsulation, while a derived class is used for inheritance

d) A derived class is used for encapsulation, while a base class is used for inheritance

Answer: b) A base class is an existing class, while a derived class is a new class

1. What is the purpose of polymorphism in C++?

a) To use a single function for multiple purposes

b) To use multiple functions for a single purpose

c) To use a single variable for multiple purposes

d) To use multiple variables for a single purpose

Answer: a) To use a single function for

Lecture#28:

Here are 30 MCQs with accurate answers

1. Which operator is used for dynamic memory allocation in C++?

a) new

b) malloc

c) calloc

d) realloc

Answer: a) new

1. What is the purpose of the new operator in C++?

a) To allocate memory at compile time

b) To allocate memory at runtime

c) To deallocate memory

d) To initialize objects

Answer: b) To allocate memory at runtime

1. What is the difference between new and malloc()?

a) new is an operator, malloc() is a function

b) new allocates memory on the stack, malloc() on the heap

c) new is used for objects, malloc() for primitive data types

d) new returns a pointer, malloc() returns an integer

Answer: a) new is an operator, malloc() is a function

1. What is the purpose of the delete operator in C++?

a) To allocate memory

b) To deallocate memory

c) To initialize objects

d) To destroy objects

Answer: b) To deallocate memory

1. When is the destructor called in a C++ program?

a) When an object is created

b) When an object is destroyed

c) When memory is allocated

d) When memory is deallocated

Answer: b) When an object is destroyed

1. What is the purpose of a constructor in a C++ class?

a) To allocate memory

b) To deallocate memory

c) To initialize objects

d) To destroy objects

Answer: c) To initialize objects

1. What is the difference between a class and a structure in C++?

a) A class is public by default, a structure is private by default

b) A class is private by default, a structure is public by default

c) A class can have member functions, a structure cannot

d) A class cannot have member functions, a structure can

Answer: b) A class is private by default, a structure is public by default

1. What is the purpose of the public keyword in a C++ class?

a) To make members private

b) To make members public

c) To make members protected

d) To make members inaccessible

Answer: b) To make members public

1. What is the purpose of the private keyword in a C++ class?

a) To make members public

b) To make members private

c) To make members protected

d) To make members inaccessible

Answer: b) To make members private

1. What is the difference between a pointer and a reference in C++?

a) A pointer stores a memory address, a reference stores a value

b) A pointer stores a value, a reference stores a memory address

c) A pointer is used for primitive data types, a reference is used for objects

d) A pointer is used for objects, a reference is used for primitive data types

Answer: a) A pointer stores a memory address, a reference stores a value

1. What is the purpose of the new operator in a C++ constructor?

a) To allocate memory for the object

b) To deallocate memory for the object

c) To initialize the object's members

d) To destroy the object

Answer: a) To allocate memory for the object

1. What is the purpose of the delete operator in a C++ destructor?

a) To allocate memory for the object

b) To deallocate memory for the object

c) To initialize the object's members

d) To destroy the object

Answer: b) To deallocate memory for the object

1. What is a memory leak in C++?

a) When memory is allocated but not deallocated

b) When memory is deallocated but not allocated

c) When an object is created but not destroyed

d) When an object is destroyed but not created

Answer: a) When memory is allocated but not deallocated

1. What is the purpose of a destructor in a C++ class?

a) To allocate memory for the object

b) To deallocate memory for the object

c) To initialize the object's members

d) To destroy the object

Answer: b) To deallocate memory for the object

1. What is the difference between a shallow copy and a deep copy in C++?

a) A shallow copy copies the memory address, a deep copy copies the value

b) A shallow copy copies the value, a deep copy copies the memory address

c)There was a problem generating a response. Please try again later.

1. Can new be called from some location other than a constructor?

a) No

b) Yes

Answer: b) Yes

2. What is the purpose of the delete operator in C++?

a) To allocate memory

b) To deallocate memory

Answer: b) To deallocate memory

3. Why do we prefer to use new and delete operators in C++?

a) Because they are designed to work with classes and objects

b) Because they are faster than malloc() and free()

Answer: a) Because they are designed to work with classes and objects

4. Where is the main() function usually located in a C++ program?

a) Before the class definitions

b) After the class definitions

Answer: b) After the class definitions

5. What is class abstraction?

a) Exposing the implementation details of a class to the users

b) Hiding the implementation details of a class from the users

Answer: b) Hiding the implementation details of a class from the users

6. What is meant by sending a message to an object in C++?

a) Calling a function

b) Creating an object

Answer: a) Calling a function

7. What is the purpose of overloading operators in C++?

a) To extend the language

b) To reduce code size

Answer: a) To extend the language

8. Can we use C functions like malloc() and free() in C++?

a) No

b) Yes

Answer: b) Yes

9. What is the advantage of using classes in C++?

a) We can extend the language

b) We can reduce code size

Answer: a) We can extend the language

10. What is the purpose of a constructor in a class?

a) To deallocate memory

b) To initialize objects

Answer: b) To initialize objects

Lecture 29:

[11:30 PM, 7/8/2024] Meta AI: Here are 30 MCQs with accurate answers based on the provided text:

Program-based MCQs

1. What is the output of the program?

```
class myClass {
  friend void increment(myClass *, int);
  private:
    int topSecret;
  public:
    void display() { cout << topSecret; }
};

void increment(myClass *a, int i) { a->topSecret += i; }

int main() {
  myClass x;
  x.display();
  increment(&x, 10);
```

```
  x.display();
  return 0;
}
```

a) 100 110

b) 110 100

c) 100 100

d) 110 110

Answer: a) 100 110

1. What is the output of the program?

```
class myClass1 {
  friend void addBoth(myClass1, myClass2);
  private:
    int topSecret;
  public:
    void display() { cout << topSecret; }
};

class myClass2 {
  friend void addBoth(myClass1, myClass2);
  private:
    int topSecret;
  public:
    void display() { cout << topSecret; }
};

void addBoth(myClass1 a, myClass2 b) {
```

```
  cout << a.topSecret + b.topSecret;
}

int main() {
  myClass1 a;
  myClass2 b;
  a.display();
  b.display();
  addBoth(a, b);
  return 0;
}
```

a) 300

b) 200

c) 100

d) 400

Answer: a) 300

Theory-based MCQs

1. What is the purpose of friend functions in C++?

a) To encapsulate data and functions

b) To provide access to private data members

c) To declare a class as a friend

d) To inherit properties from another class

Answer: b) To provide access to private data members

1. Which of the following is a characteristic of friend functions?

a) They are members of the class

b) They can access private data members

c) They are declared in the public section

d) They are transitive

Answer: b) They can access private data members

1. What is the difference between a friend function and a member function?

a) A friend function is a member of the class, while a member function is not

b) A friend function can access private data members, while a member function cannot

c) A friend function is declared in the public section, while a member function is declared in the private section

d) A friend function is used for encapsulation, while a member function is used for inheritance

Answer: b) A friend function can access private data members, while a member function cannot

1. When is a friend function used?

a) When we want to encapsulate data and functions

b) When we want to provide access to private data members

c) When we want to declare a class as a friend

d) When we want to inherit properties from another class

Answer: b) When we want to provide access to private data members

1. What is the purpose of friend classes in C++?

a) To encapsulate data and functions

b) To provide access to private data members

c) To declare a class as a friend

d) To inherit properties from another class

Answer: c) To declare a class as a friend

1. Which of the following is a characteristic of friend classes?

a) They are members of the class

b) They can access private data members

c) They are declared in the public section

d) They are transitive

Answer: b) They can access private data members

1. What is the difference between a friend class and a friend function?

a) A friend class is a member of the class, while a friend function is not

b) A friend class can access private data members, while a friend function cannot

c) A friend class is declared in the public section, while a friend function is declared in the private section

d) A friend class is used for encapsulation, while a friend function is used for inheritance

Answer: b) A friend class can access private data members, while a friend function cannot

1. When is a friend class used?

a) When we want to encapsulate data and functions

b) When we want to provide access to private data members

c) When we want to declare a class as a friend

d) When we want to inherit properties from another class

Answer: c) When we want to declare a class as a friend

1. What is the purpose of the destructor in a class?

a) To allocate memory

b) To deallocate memory

Answer: b) To deallocate memory

2. Can we write our own class for complex numbers in C++?

a) No

b) Yes

Answer: b) Yes

3. What is the benefit of using classes to extend the language?

a) We can use only existing data types

b) We can create our own data types

Answer: b) We can create our own data types

4. What is the difference between new and malloc()?

a) new is used for classes, malloc() is used for structures

b) new is used for arrays, malloc() is used for single objects

Answer: a) new is used for classes, malloc() is used for structures

5. What is the purpose of the main() function in a C++ program?

a) To define classes

b) To start program execution

Answer: b) To start program execution

6. What is the advantage of using new and delete operators over malloc() and free()?

a) They are faster

b) They are safer and easier to use

Answer: b) They are safer and easier to use

7. Can we use delete operator to deallocate memory allocated by malloc()?

a) Yes

b) No

Answer: b) No

8. What is the purpose of class abstraction?

a) To show the implementation details of a class

b) To hide the implementation details of a class

Answer: b) To hide the implementation details of a class

9. What is the benefit of using classes in C++?

a) We can write more complex code

b) We can write reusable and modular code

Answer: b) We can write reusable and modular code

10. What is the difference between a class and a structure?

a) A class is used for built-in data types, a structure is used for user-defined data types

b) A class is used for user-defined data types, a structure is used for built-in data types

Answer: b) A class is used for user-defined data types, a structure is used for built-in data types

Lecture#30:

Theory-based MCQs

1. What is a reference in C++?

a) Another name for a pointer

b) A synonym for a variable

c) A new data type

d) A function call

Answer: b) A synonym for a variable

1. What is the purpose of references in C++?

a) To implement call by value

b) To implement call by reference

c) To create a new data type

d) To create a new function

Answer: b) To implement call by reference

1. What is the difference between a reference and a pointer?

a) A reference is a synonym, while a pointer is an address

b) A reference is an address, while a pointer is a synonym

c) A reference is used for integers, while a pointer is used for characters

d) A reference is used for characters, while a pointer is used for integers

Answer: a) A reference is a synonym, while a pointer is an address

1. Can we do arithmetic with references?

a) Yes

b) No

c) Sometimes

d) Never

Answer: b) No

1. Must reference variables be initialized when declared?

a) Yes

b) No

c) Sometimes

d) Never

Answer: a) Yes

1. What happens if we return a reference to a local variable from a function?

a) It becomes a global variable

b) It becomes a static variable

c) It becomes a dangling reference

d) It becomes a pointer

Answer: c) It becomes a dangling reference

1. What is the benefit of using references over pointers?

a) References are faster

b) References are safer

c) References are more efficient

d) References are more elegant

Answer: d) References are more elegant

1. Can we use references with functions?

a) Yes

b) No

c) Sometimes

d) Never

Answer: a) Yes

1. What is the purpose of the & symbol in references?

a) To take the address of a variable

b) To declare a reference

c) To initialize a reference

d) To assign a value to a reference

Answer: b) To declare a reference

1. Can we use references with arrays?

a) Yes

b) No

c) Sometimes

d) Never

Answer: a) Yes

Program-based MCQs

1. What is the output of the following program?

```
int x = 5;
int &ref = x;
cout << ref;
```

a) 5

b) 10

c) 0

d) Error

Answer: a) 5

1. What is the output of the following program?

```
int x = 5;
int *ptr = &x;
cout << *ptr;
```

a) 5

b) 10

c) 0

d) Error

Answer: a) 5

1. What is the output of the following program?

```
int x = 5;
int &ref = x;
ref = 10;
cout << x;
```

a) 5

b) 10

c) 0

d) Error

Answer: b) 10

1. What is the output of the following program?

```
int x = 5;
```

```
int &ref = x;
int y = ref;
cout << y;
```

a) 5

b) 10

c) 0

d) Error

Answer: a)5

Lecture#31:

1. What is operator overloading in C++?

a) Redefining operators for built-in data types

b) Redefining operators for user-defined data types

c) Creating new operators

d) Deleting existing operators

Answer: b) Redefining operators for user-defined data types

1. Which operators can be overloaded in C++?

a) Only arithmetic operators

b) Only comparison operators

c) All operators except . , :: , .* , and ?

d) All operators

Answer: c) All operators except . , :: , .* , and ?

1. What is the purpose of operator overloading?

a) To increase code complexity

b) To decrease code readability

c) To allow user-defined data types to work with operators

d) To replace existing operators

Answer: c) To allow user-defined data types to work with operators

1. What is the syntax for overloading an operator?

a) return-type operator operator-symbol (parameter-list)

b) return-type operator-symbol operator (parameter-list)

c) operator return-type operator-symbol (parameter-list)

d) operator-symbol return-type operator (parameter-list)

Answer: a) return-type operator operator-symbol (parameter-list)

1. Which operators cannot be overloaded?

a) +, -, _, /

b) ., ::, .*, ?

c) =,

d) All of the above

Answer: b) ., ::, .*, ?

1. What is the advantage of operator overloading?

a) Increased code complexity

b) Decreased code readability

c) Ability to work with user-defined data types

d) Ability to replace existing operators

Answer: c) Ability to work with user-defined data types

1. Can we overload operators for built-in data types?

a) Yes

b) No

c) Sometimes

d) Never

Answer: b) No

1. What is the purpose of the operator keyword in C++?

a) To define a new operator

b) To overload an existing operator

c) To delete an existing operator

d) To redefine a built-in operator

Answer: b) To overload an existing operator

1. Which operators are already overloaded for primitive types in C++?

a) +, -, _, /

b) =,

c) ., ::, .*, ?

d) All of the above

Answer: a) +, -, *, /

1. What is the difference between a member operator and a friend operator?

a) Member operator is used for built-in types, friend operator is used for user-defined types

b) Member operator is used for user-defined types, friend operator is used for built-in types

c) Member operator is used for private members, friend operator is used for public members

d) Member operator is used for public members, friend operator is used for private members

Answer: b) Member operator is used for user-defined types, friend operator is used for built-in types

1. What is the purpose of the + operator function in the Complex class?

a) To add two complex numbers

b) To subtract two complex numbers

c) To multiply two complex numbers

d) To divide two complex numbers

Answer: a) To add two complex numbers

1. How many operands does the + operator require?

a) 1

b) 2

c) 3

d) 4

Answer: b) 2

1. What is the purpose of the temp object in the + operator function?

a) To store the result of the addition

b) To store the operands of the addition

c) To store the result of the subtraction

d) To store the operands of the subtraction

Answer: a) To store the result of the addition

1. What is the purpose of the return statement in the + operator function?

a) To return the result of the addition

b) To return the operands of the addition

c) To return the result of the subtraction

d) To return the operands of the subtraction

Answer: a) To return the result of the addition

1. Can we overload the + operator for a class with a single operand?

a) Yes

b) No

c) Sometimes

d) Never

Answer: b) No

1. What is the purpose of the += operator?

a) To add and assign the result to the left operand

b) To subtract and assign the result to the left operand

c) To multiply and assign the result to the left operand

d) To divide and assign the result to the left operand

Answer: a) To add and assign the result to the left operand

1. How many operands does the += operator require?

a) 1

b) 2

c) 3

d) 4

Answer: b) 2

Lecture#32:

 Which of the following operators cannot be overloaded in C++?

- A) +
- B) -
- C) ::
- D) []

Answer: C) ::

 What is the return type of an overloaded binary operator function?

- A) The type of the second operand
- B) The type of the first operand
- C) The type specified in the operator overloading declaration
- D) void

Answer: C) The type specified in the operator overloading declaration

 How do you overload the increment operator (++) to increment a value in the Date class?

- A) Define it as a member function
- B) Define it as a friend function
- C) Define it as a static member function
- D) Define it in the Date class but outside any method

Answer: A) Define it as a member function

 Which operator can be overloaded as both a member and a non-member function?

- A) +
- B) =
- C) <<
- D) ::

Answer: A) +

 How many arguments can a member function for overloading a binary operator have?

- A) 0
- B) 1
- C) 2
- D) Any number

Answer: B) 1

 What is the primary benefit of overloading operators?

- A) To allow user-defined operations on class objects
- B) To increase the execution speed of operations
- C) To reduce memory usage
- D) To enforce security constraints

Answer: A) To allow user-defined operations on class objects

 Which operator cannot be overloaded in C++?

- A) []
- B) ()
- C) ,
- D) .

Answer: D) .

 If you want to overload the == operator for a class, what should you ensure?

- A) It should be a member function
- B) It should be a non-member function
- C) It should return a boolean value
- D) It should not modify any member variables

Answer: C) It should return a boolean value

 In which scenario would you typically use the friend function to overload an operator?

- A) When the operator needs access to private members of a class
- B) When overloading unary operators
- C) When defining a default constructor
- D) When creating a derived class

Answer: A) When the operator needs access to private members of a class

 Which of the following operators cannot be overloaded?

- A) =
- B) -
- C) <<
- D) ++

Answer: A) =

 When overloading the << operator for output, which function signature is correct?

- A) ostream& operator<<(ostream& os, const MyClass& obj);
- B) ostream operator<<(ostream& os, const MyClass& obj);
- C) ostream& operator<<(const MyClass& obj, ostream& os);
- D) ostream operator<<(const MyClass& obj, ostream& os);

Answer: A) ostream& operator<<(ostream& os, const MyClass& obj);

 When overloading the == operator, what should the function return?

- A) The object itself
- B) An integer
- C) A boolean value
- D) The type of the object

Answer: C) A boolean value

 Which operator must always be overloaded as a member function?

- A) =
- B) <<
- C) []
- D) ,

Answer: A) =

 What is the result of overloading the * operator in a class?

- A) The operator performs multiplication on class objects.
- B) The operator is not supported for overloading.
- C) The operator performs a bitwise operation.
- D) The operator performs string concatenation.

Answer: A) The operator performs multiplication on class objects.

 In operator overloading, which of the following operators should be defined as a non-member function?

- A) Unary operators
- B) Binary operators that need to access private members
- C) Stream insertion and extraction operators
- D) Assignment operators

Answer: C) Stream insertion and extraction operators

 How do you overload the [] operator in a class?

- A) As a member function that takes an integer index
- B) As a non-member function that takes an integer index
- C) As a member function that takes a reference to another class
- D) As a non-member function that takes a reference to another class

Answer: A) As a member function that takes an integer index

 Can you overload the `new` and `delete` operators in C++?

- A) No, these operators cannot be overloaded.
- B) Yes, but only for classes that allocate memory dynamically.
- C) Yes, you can overload `new` and `delete` operators globally.
- D) Yes, you can overload `new` and `delete` operators within a class.

Answer: D) Yes, you can overload `new` and `delete` operators within a class.

 What is the main advantage of overloading operators for user-defined classes?

- A) It simplifies code and makes it more readable.
- B) It reduces the runtime complexity of operations.
- C) It improves memory allocation efficiency.
- D) It provides access to private and protected members.

Answer: A) It simplifies code and makes it more readable.

 What is the result of overloading the unary - operator in a class?

- A) The operator negates the value of the object.
- B) The operator performs a bitwise AND operation.
- C) The operator performs addition.
- D) The operator performs multiplication.

Answer: A) The operator negates the value of the object.

 How can you overload the - operator to perform subtraction for a class?

- A) Define it as a non-member function that takes two objects as parameters.
- B) Define it as a member function that takes one object as a parameter.
- C) Define it as a member function that takes two objects as parameters.
- D) Define it as a non-member function that takes one object as a parameter.

Answer: B) Define it as a member function that takes one object as a parameter.

 What should you do if you want to overload the + operator and maintain the original

object unchanged?

- A) Define it as a non-member function.
- B) Define it as a member function.
- C) Use a constant member function.
- D) Return a new object by value.

Answer: D) Return a new object by value.

 Can the assignment operator (=) be overloaded in C++?

- A) Yes, but it must always be a member function.
- B) No, it cannot be overloaded.
- C) Yes, it can be a member or non-member function.
- D) Yes, but it should be used only with fundamental types.

Answer: A) Yes, but it must always be a member function.

 Which operator cannot be overloaded as a member function?

- A) ++
- B) -
- C) <<
- D) ::

Answer: D) ::

 How should the == operator be defined to ensure it compares two objects of a class correctly?

- A) As a non-member function that takes two objects as parameters.
- B) As a member function that takes one object as a parameter.
- C) As a static member function.
- D) As a friend function that takes two objects as parameters.

Answer: D) As a friend function that takes two objects as parameters.

 When overloading the [] operator, what is the typical return type?

- A) void
- B) A reference to the type of element stored
- C) An integer
- D) A boolean

Answer: B) A reference to the type of element stored

 Which of the following is a valid use of the != operator overload?

- A) To check if two objects are not equal
- B) To assign one object to another
- C) To output an object to a stream
- D) To increment an object's value

Answer: A) To check if two objects are not equal

 What must you do when overloading the = operator to ensure it performs a deep copy?

- A) Use the default assignment operator.
- B) Explicitly define the assignment operator to handle deep copying of resources.
- C) Use a constructor to handle the copying.
- D) Only copy the basic data members.

Answer: B) Explicitly define the assignment operator to handle deep copying of resources.

 Which of the following operators is usually overloaded as a non-member function?

- A) =
- B) ==
- C) <<
- D) ++

Answer: C) <<

 What is the result of overloading the * operator in a class?

- A) You can define how the multiplication operator behaves for class objects.
- B) You can define how bitwise operations are performed.
- C) It enables multiplication with only numeric types.
- D) It performs memory management operations.

Answer: A) You can define how the multiplication operator behaves for class objects.

 How should you define the post-increment operator (++) in a class to ensure it returns the original value?

- A) Define it as a member function and return a copy of the current object.
- B) Define it as a non-member function and return a reference to the original object.
- C) Define it as a member function and return a reference to the updated object.
- D) Define it as a friend function and return a new object.

Answer: A) Define it as a member function and return a copy of the current object.

Lecture # 33:

1. **What is the purpose of operator overloading in C++?**
   - A. To create new operators
   - B. To modify the behavior of existing operators for user-defined types
   - C. To make operators faster
   - D. To prevent operator use in classes

   Answer: B

2. **Which operators cannot be overloaded in C++?**
   - A. + and -
   - B. == and !=
   - C. . (member access) and :: (scope resolution)
   - D. * and /

   Answer: C

3. **What is the return type of an overloaded assignment operator (operator=) in C++?**
   - A. void
   - B. bool
   - C. The class type
   - D. A reference to the class type

   Answer: D

4. **Which operator can be overloaded to allow for custom behavior with array indexing?**
   - A. +
   - B. *
   - C. []
   - D. &

   Answer: C

5. **When overloading the << operator for output streams, what should be the return type?**
   - A. void
   - B. The class type
   - C. ostream&
   - D. istream&

   Answer: C

6. In C++, what does the `this` pointer represent?
    - A. A pointer to the class definition
    - B. A pointer to the base class
    - C. A pointer to the current object instance
    - D. A pointer to the next object in memory

    Answer: C

7. How can you prevent a class from being copied using the assignment operator?
    - A. By defining a private copy constructor
    - B. By defining a private assignment operator
    - C. By deleting the copy constructor
    - D. By defining a public assignment operator

    Answer: B

8. What does the `operator++` function typically do if implemented as a member function?
    - A. Increments the value of the current object
    - B. Returns the value of the current object
    - C. Decrements the value of the current object
    - D. Returns a boolean indicating success

    Answer: A

9. What is the correct way to implement the `operator==` for a class?
    - A. It should return `void` and compare the members
    - B. It should return a boolean indicating whether the members are equal
    - C. It should modify the class members to be equal
    - D. It should use a static member function

    Answer: B

10. What happens if you do not define a copy assignment operator in a class?
    - A. The compiler will automatically generate a default assignment operator
    - B. The program will not compile
    - C. The copy assignment operator will be deleted
    - D. The program will produce a runtime error

    Answer: A

11. How should you handle self-assignment in an assignment operator?
    - A. Ignore it; self-assignment does not need special handling
    - B. Check if the current object is the same as the object being assigned and do nothing if true
    - C. Always delete the current object before assignment

- D. Always create a new object before assignment

Answer: B

12. **Which operator is typically overloaded to provide custom behavior for relational operations?**
    - A. +
    - B. ==
    - C. <<
    - D. []

    Answer: B

13. **What is a common practice when implementing the operator<< for a class?**
    - A. To modify the state of the object
    - B. To return a reference to the output stream
    - C. To return a new instance of the class
    - D. To print the class members directly

    Answer: B

14. **In operator overloading, how can you overload the () operator?**
    - A. By defining a member function with the name operator()
    - B. By defining a global function named operator()
    - C. By defining a static member function named operator()
    - D. By defining a constructor with no arguments

    Answer: A

15. **Which operator should be used to allow an object to be used in a logical negation context?**
    - A. !
    - B. ~
    - C. -
    - D. +

    Answer: A

MCQs on Type Conversions

16. **How is a conversion constructor defined in a class?**
    - A. By using the explicit keyword
    - B. By using a constructor with a single parameter
    - C. By using a static member function
    - D. By using a conversion operator

Answer: B

17. What is the purpose of a conversion operator in a class?
    - A. To convert the class object to another data type
    - B. To overload arithmetic operators
    - C. To modify the class members
    - D. To create new operators

Answer: A

18. What is the syntax for defining a conversion operator in C++?
    - A. operator Type()
    - B. Type operator()
    - C. Type operator Type()
    - D. Type() operator

Answer: A

19. What is the effect of the explicit keyword when used with a conversion constructor?
    - A. It allows implicit conversions to be performed
    - B. It prevents implicit conversions, requiring explicit conversion calls
    - C. It allows implicit conversion in all cases
    - D. It makes the constructor public

Answer: B

20. How can a class provide a conversion to int?
    - A. By defining a member function operator int()
    - B. By defining a member function int operator()
    - C. By defining a constructor that takes an int
    - D. By defining a static function int conversion()

Answer: A

21. What is the result of a conversion operator if it returns void?
    - A. It will compile and return nothing
    - B. It will produce a compile-time error
    - C. It will return the class object
    - D. It will not perform any conversion

Answer: B

22. When defining a conversion operator for a class, what must it return?
    - A. A reference to the class type
    - B. A pointer to the class type

- C. The target type of the conversion
- D. void

Answer: C

23. What happens if you overload the + operator in a class?
    - A. It modifies the internal state of the class
    - B. It allows instances of the class to be added together using the + operator
    - C. It prevents addition of instances of the class
    - D. It affects the class's memory allocation

    Answer: B

24. In C++, how do you define a member function that returns a reference to the object itself?
    - A. Type& memberFunction()
    - B. Type* memberFunction()
    - C. Type memberFunction()
    - D. void memberFunction()

    Answer: A

25. Which operator is overloaded to provide custom behavior for the unary minus (-) operation?
    - A. operator+
    - B. operator-
    - C. operator*
    - D. operator/

    Answer: B

26. What should be checked to prevent memory leaks when overloading the assignment operator?
    - A. Whether the object being assigned is null
    - B. Whether the current object is being assigned to itself
    - C. Whether the object has already been deleted
    - D. Whether the current object is empty

    Answer: B

27. How do you overload the * operator to provide custom behavior for multiplication?
    - A. By defining a global function named operator*
    - B. By defining a member function named operator*
    - C. By defining a static function named operator*

- D. By defining a non-member function named operator*

Answer: B

28. **What is the purpose of the operator~ in C++ when overloading bitwise NOT?**
    - A. To invert all the bits of the object
    - B. To create a new instance of the object
    - C. To modify the state of the object
    - D. To compare the object with another object

    Answer: A

29. **Which operator can be overloaded to allow objects of a class to be used in conditional expressions?**
    - A. +
    - B. -
    - C. ==
    - D. !

    Answer: D

30. **What does the operator[] function allow you to do when overloaded?**
    - A. Modify the index of the array
    - B. Access elements in a class as if it were an array
    - C. Create new arrays
    - D. Delete elements from the array

    Answer: B

Lecture#34:

1. **Which constructor is called when creating an array of objects without explicit initialization?**
    - A. Parameterized constructor
    - B. Destructor
    - C. Default constructor
    - D. Copy constructor

    Answer: C. Default constructor

2. **What happens when you initialize only the first two elements of an array of objects?**
    - A. Only the first two elements are initialized, and the rest are not constructed.
    - B. The first two elements are initialized with provided values; the rest use the default constructor.

- C. All elements are initialized with provided values.
- D. Compilation error occurs.

Answer: B. The first two elements are initialized with provided values; the rest use the default constructor.

3. **In the `Date` class example, what happens when the statement `Date myDate[10]` is executed?**
   - A. 10 objects are created using the parameterized constructor.
   - B. 10 objects are created using the default constructor.
   - C. Only one object is created.
   - D. Compilation error occurs.

   Answer: B. 10 objects are created using the default constructor.

4. **What does the following initialization do: `Date yourDate[3] = { Date(10, 24, 1980), Date(06, 14, 1985), Date(07, 09, 1986) };`?**
   - A. Calls the default constructor for all elements.
   - B. Initializes the first three elements with specified values and uses the default constructor for any remaining elements.
   - C. Initializes all elements with specified values.
   - D. Causes a runtime error.

   Answer: B. Initializes the first three elements with specified values and uses the default constructor for any remaining elements.

5. **Which constructor is called when creating an array of `String` objects with this initialization: `String message[10] = { "First line of message\n", "Second line of message\n", String("Third line of message\n"), String() };`?**
   - A. Default constructor for all elements.
   - B. Constructor with one `char*` parameter for the first two elements, and parameterized constructors for the third and fourth elements.
   - C. Parameterized constructor for all elements.
   - D. Only the first element is initialized.

   Answer: B. Constructor with one `char*` parameter for the first two elements, and parameterized constructors for the third and fourth elements.

6. **How does dynamic memory allocation for an array of objects work?**
   - A. The `new` operator allocates space for the array and calls the default constructor for each element.
   - B. The `new` operator allocates space and initializes elements to zero.
   - C. The `new` operator only allocates space; initialization must be done manually.
   - D. The `new` operator does not allocate space for arrays.

Answer: A. The new operator allocates space for the array and calls the default constructor for each element.

7. What is the correct way to delete a dynamically allocated array of objects?
   - A. delete arrayPointer;
   - B. delete [] arrayPointer;
   - C. free(arrayPointer);
   - D. delete arrayPointer[];

   Answer: B. delete [] arrayPointer;

8. What happens if you use delete instead of delete [] for a dynamically allocated array?
   - A. All objects in the array are deleted correctly.
   - B. Only the first object in the array is deleted, leading to a memory leak.
   - C. The program will crash immediately.
   - D. The objects are deleted but not deallocated properly.

   Answer: B. Only the first object in the array is deleted, leading to a memory leak.

9. Which of the following statements about overloading new and delete operators is true?
   - A. Overloaded new and delete operators can only be used globally.
   - B. Overloaded new and delete operators can be used globally or as member functions.
   - C. Overloading new and delete operators is not allowed in C++.
   - D. Overloaded new and delete operators can only be used for primitive data types.

   Answer: B. Overloaded new and delete operators can be used globally or as member functions.

10. What is the purpose of overloading the new operator in embedded and real-time systems?
    - A. To allocate larger blocks of memory.
    - B. To fragment memory for efficient use.
    - C. To ensure memory allocation takes constant time.
    - D. To deallocate memory more efficiently.

    Answer: C. To ensure memory allocation takes constant time.

11. What does the following overloaded new operator do: void * operator new(size_t size) { void * rtn = calloc(1, size); return rtn; }?
    - A. Allocates memory and initializes it to zero.
    - B. Allocates memory and sets it to one.
    - C. Allocates memory without initializing it.

- D. Allocates memory and sets it to null.

Answer: A. Allocates memory and initializes it to zero.

12. **In the provided example, what does the `Name` class's custom `new` operator do?**
    - A. Allocates memory from a pool and marks it as used.
    - B. Allocates memory on the heap.
    - C. Allocates memory using `calloc`.
    - D. Allocates memory using `malloc`.

    Answer: A. Allocates memory from a pool and marks it as used.

13. **When overloading `operator[]` in a class, what is a common purpose?**
    - A. To manage memory allocation.
    - B. To provide array-like access with bounds checking.
    - C. To modify the destructor behavior.
    - D. To initialize array elements.

    Answer: B. To provide array-like access with bounds checking.

14. **What does the `operator[]` function in the `IntArray` class example return?**
    - A. A value.
    - B. A reference to an integer.
    - C. A pointer to an integer.
    - D. An object of type `IntArray`.

    Answer: B. A reference to an integer.

15. **What is a potential issue when not overloading `operator[]` to include bounds checking?**
    - A. The program will run faster.
    - B. The compiler will automatically handle out-of-bounds errors.
    - C. The program may access memory outside the allocated array, leading to undefined behavior.
    - D. The program will always throw an exception.

    Answer: C. The program may access memory outside the allocated array, leading to undefined behavior.

16. **In the `Name` class example, what is the role of the `inuse` array?**
    - A. To keep track of the number of objects created.
    - B. To manage the allocation and deallocation status of memory in the pool.
    - C. To store the names of the `Name` objects.
    - D. To initialize memory to zero.

    Answer: B. To manage the allocation and deallocation status of memory in the

pool.

17. What happens if the new operator fails to allocate memory in the Name class example?
    - A. The program crashes.
    - B. It returns a null pointer.
    - C. It returns an integer value.
    - D. It reuses previously allocated memory.

    Answer: B. It returns a null pointer.

18. What does the delete operator do in the Name class example?
    - A. It deallocates memory and marks it as unused.
    - B. It simply deallocates memory.
    - C. It initializes the memory to zero.
    - D. It does nothing.

    Answer: A. It deallocates memory and marks it as unused.

19. What is the expected output when using the following code to create an array of integers with new and delete: int *ip = new int[10]; ... delete [] ip;?
    - A. The memory is deallocated and the program terminates successfully.
    - B. Only the first integer is deleted.
    - C. The program will crash due to incorrect deallocation.
    - D. The integers in the array are initialized to zero.

    Answer: A. The memory is deallocated and the program terminates successfully.

20. When you overload operator new and operator delete globally, what happens?
    - A. Only specific classes use the overloaded operators.
    - B. All new and delete operations use the global versions.
    - C. It affects the new and delete operators only for arrays.
    - D. It disables the use of the standard new and delete operators.

    Answer: B. All new and delete operations use the global versions.

16. What is the output of the following code when declaring and initializing a Date array?

cpp
Copy code
Date myDates[10] = { Date(09, 03, 1970), Date(08, 23, 1974) };

a) Only the first two Date objects are initialized; the rest are default-initialized.
b) All ten Date objects are initialized with the date 09/03/1970.
c) Only the first Date object is initialized, the rest are default-initialized.
d) The code will not compile due to incorrect initialization.

Answer: a) Only the first two `Date` objects are initialized; the rest are default-initialized.

---

17. Which constructor is used when creating an array of `Date` objects without any initialization?

a) Default constructor
b) Parameterized constructor
c) Destructor
d) The compiler does not call any constructor.

Answer: a) Default constructor

---

18. In the `String` class example, which constructor is used for the initialization of the array of `String` objects?

a) Constructor with one `char*` parameter
b) Parameterless constructor
c) Copy constructor
d) Default constructor

Answer: a) Constructor with one `char*` parameter

---

19. What happens if you attempt to delete an array allocated with `new` using `delete` without brackets?

a) It will deallocate the entire array correctly.
b) It will call the destructor for each object and deallocate the array.
c) It will only delete the first object and cause a memory leak for the rest.
d) It will not compile.

Answer: c) It will only delete the first object and cause a memory leak for the rest.

---

20. How should you correctly delete an array of objects allocated with `new`?

a) `delete array;`
b) `delete array[];`
c) `delete [] array;`
d) `array.delete();`

Answer: c) `delete [] array;`

---

21. What is the purpose of overloading the `new` and `delete` operators in C++?

a) To customize memory allocation and deallocation behavior
b) To increase the execution speed of the program
c) To handle arithmetic operations on objects
d) To optimize the compilation time

Answer: a) To customize memory allocation and deallocation behavior

---

22. When overloading the `new` operator, what should it return?

a) `int`
b) `void*`
c) `char*`
d) `bool`

Answer: b) `void*`

---

23. When overloading the `delete` operator, what parameter type is expected?

a) `int*`
b) `char*`
c) `void*`
d) `float*`

Answer: c) `void*`

---

24. If you overload the `new` operator at a global level, how will it affect the `new` operator for specific classes?

a) It will override the `new` operator for all classes.
b) It will only affect the `new` operator for the specific class.
c) It will not affect any `new` operator behavior.
d) It will cause the `new` operator to behave unpredictably.

Answer: a) It will override the `new` operator for all classes.

---

25. What should you do if you need to deallocate memory for an array of objects allocated with `new`?

a) Use `delete array;`
b) Use `delete [] array;`

c) Use array.delete();
d) Use delete array[];

Answer: b) Use delete [] array;

---

26. What happens if you do not specify array brackets in the delete operator for an array allocated with new?

a) The memory is correctly deallocated.
b) Only the first object is deallocated, causing a memory leak for the rest.
c) The program will crash.
d) The delete operator will not compile.

Answer: b) Only the first object is deallocated, causing a memory leak for the rest.

---

27. In the provided Name class example, what does the operator new do?

a) Allocates memory from the system heap.
b) Allocates memory from a custom memory pool.
c) Initializes memory with zeros.
d) Allocates memory and calls the constructor.

Answer: b) Allocates memory from a custom memory pool.

---

28. What does the operator delete function in the Name class example do?

a) Deallocates memory using the system heap.
b) Marks the memory as unused in the custom memory pool.
c) Calls the destructor for each object before deallocating memory.
d) Deallocates memory and initializes it to zero.

Answer: b) Marks the memory as unused in the custom memory pool.

---

29. In the IntArray class example, what is the purpose of the overloaded operator[]?

a) To provide array indexing with boundary checking
b) To sort the array elements
c) To display the contents of the array
d) To initialize the array elements

Answer: a) To provide array indexing with boundary checking

30. What does the operator[] function return in the IntArray class?

a) An int
b) A reference to an int
c) A pointer to an int
d) A boolean value

Answer: b) A reference to an int

Lecture#35:

. What does the cin object in C++ do?

- A) It outputs data to the screen.
- B) It reads data from the screen.
- C) It reads data from the keyboard.
- D) It writes data to a file.
- **Answer:** C) It reads data from the keyboard.

What is the primary function of the cout object in C++?

- A) It reads data from the keyboard.
- B) It writes data to a file.
- C) It outputs data to the screen.
- D) It manages file streams.
- **Answer:** C) It outputs data to the screen.

Which header file must be included to use cin and cout?

- A) fstream.h
- B) stdio.h
- C) iostream.h
- D) iomanip.h
- **Answer:** C) iostream.h

What is the purpose of the << operator in C++ streams?

- A) It extracts data from the stream.
- B) It inserts data into the stream.
- C) It flushes the stream.
- D) It closes the stream.
- **Answer:** B) It inserts data into the stream.

What does the >> operator do with cin?

- A) It inserts data into the stream.
- B) It extracts data from the stream.
- C) It formats the output.
- D) It closes the stream.
- **Answer:** B) It extracts data from the stream.

 Which object is used for unbuffered output?

- A) `cout`
- B) `cerr`
- C) `clog`
- D) `cin`
- **Answer:** B) `cerr`

 What is the function of the `endl` manipulator in C++?

- A) It reads a single character from the input.
- B) It outputs a new line and flushes the stream.
- C) It writes data to a file.
- D) It formats the output.
- **Answer:** B) It outputs a new line and flushes the stream.

 Which function in `cin` reads a single character from the input?

- A) `get()`
- B) `read()`
- C) `getline()`
- D) `put()`
- **Answer:** A) `get()`

 How does `cin.getline()` function operate?

- A) It reads a single character.
- B) It reads multiple characters up to a specified delimiter.
- C) It formats the input.
- D) It writes data to a file.
- **Answer:** B) It reads multiple characters up to a specified delimiter.

 What is the role of the `flush` function in stream handling?

- A) It closes the stream.
- B) It forces the data in the buffer to be written to the destination.
- C) It reads data from the stream.
- D) It formats the output.
- **Answer:** B) It forces the data in the buffer to be written to the destination.

 Which object is used for buffered logging output?

- A) cerr
- B) cout
- C) clog
- D) cin
- **Answer:** C) clog

 What does the put() function of cout do?

- A) It reads a single character.
- B) It writes a single character.
- C) It formats the output.
- D) It flushes the stream.
- **Answer:** B) It writes a single character.

 What will be the output of the following code if i = 5 and j = 2?

```
cpp
Copy code
int i = 5, j = 2;
cout << i / j;
```

- A) 2.5
- B) 2
- C) 5/2
- D) 7
- **Answer:** B) 2

 Which of the following statements is true about cin >> i; when a non-numeric character is entered?

- A) The program will crash.
- B) The character will be stored in i.
- C) An error state will be set for the stream.
- D) The input will be ignored.
- **Answer:** C) An error state will be set for the stream.

 What is the default delimiter for cin.getline()?

- A) Space
- B) Comma
- C) New line (\n)
- D) Tab
- **Answer:** C) New line (\n)

 Which stream object is typically used to display error messages to the user?

- A) cout
- B) cerr
- C) clog
- D) cin
- **Answer:** B) cerr

 Which of the following statements is used to format the output to three decimal places in C++?

- A) cout << fixed << setprecision(3) << value;
- B) cout << precision(3) << fixed << value;
- C) cout << setprecision(3) << value;
- D) cout << value << fixed << setprecision(3);
- **Answer:** A) cout << fixed << setprecision(3) << value;

 Which function is used to read a complete line of text including spaces in C++?

- A) cin.get()
- B) cin.read()
- C) cin.getline()
- D) cin >>
- **Answer:** C) cin.getline()

 How does `cout << "Hello " << "World";` work in terms of stream operators?

- A) It prints "Hello World" as a single string.
- B) It prints "Hello" followed by "World" on a new line.
- C) It prints "Hello" and then "World" in sequence on the same line.
- D) It prints "Hello World" with a space between Hello and World.
- **Answer:** C) It prints "Hello" and then "World" in sequence on the same line.

 In C++ streams, what does `cout << x;` do if `x` is a variable holding a float value?

- A) It prints the float value as is.
- B) It converts the float to an integer before printing.
- C) It prints the float value with default precision.
- D) It reads the float value from the user.
- **Answer:** C) It prints the float value with default precision.

 What does the `<<` operator return in the context of `cout`?

- A) The value to be printed.
- B) A reference to the `cout` object.
- C) The formatted output.
- D) The end of the stream.
- **Answer:** B) A reference to the `cout` object.

 When using `cin`, what happens if an invalid data type is entered for an integer variable?

- A) The invalid input is stored in the integer variable.
- B) The program crashes.
- C) The stream enters a fail state.
- D) The input is ignored, and the variable remains unchanged.
- **Answer:** C) The stream enters a fail state.

 Which header file provides stream manipulators like `setprecision`?

- A) `iostream.h`
- B) `fstream.h`
- C) `iomanip.h`
- D) `stdlib.h`
- **Answer:** C) `iomanip.h`

 What is the effect of `cout << flush;` in C++?

- A) It ends the current line of output.
- B) It formats the output.
- C) It forces the buffer to be written to the screen immediately.
- D) It reads input from the user.
- **Answer:** C) It forces the buffer to be written to the screen immediately.

 How can you ensure that a loop counter is displayed immediately on the screen without buffering?

- A) Use `cout` with `endl` at each iteration.
- B) Use `cerr` for displaying the loop counter.
- C) Use `cin` to output the counter value.
- D) Use `clog` for displaying the loop counter.
- **Answer:** B) Use `cerr` for displaying the loop counter.

 What is the purpose of the `cerr` object in C++?

- A) It is used for buffered error output.
- B) It is used for unbuffered error output.
- C) It is used for formatted error output.
- D) It is used for standard output.
- **Answer:** B) It

Lecture # 36.

1. . Which header file is required for using manipulators in C++?
    - A) <iostream>
    - B) <fstream>
    - C) <iomanip>
    - D) <string>

    Answer: C) <iomanip>

2. What does the cin.eof() function check for?
    - A) The stream is in good state
    - B) The end of file has been reached
    - C) The stream is in a fail state
    - D) The stream is bad

    Answer: B) The end of file has been reached

3. Which manipulator is used to set the width of the output field?
    - A) setprecision()
    - B) setw()
    - C) setfill()
    - D) flush()

    Answer: B) setw()

4. What does the endl manipulator do?
    - A) Inserts a new line character and flushes the stream
    - B) Inserts a new line character without flushing the stream
    - C) Flushes the stream without inserting a new line
    - D) Sets the width of the output field

    Answer: A) Inserts a new line character and flushes the stream

5. Which manipulator will display the output in hexadecimal format?
    - A) oct
    - B) dec
    - C) hex
    - D) base

    Answer: C) hex

6. Which function resets the stream's state flags to their default values?
    - A) cin.flush()
    - B) cin.clear()
    - C) cin.reset()
    - D) cin.eof()

Answer: B) cin.clear()

7. **What is the purpose of the setfill manipulator?**
   - A) To set the decimal precision
   - B) To specify the fill character for padding
   - C) To set the base for numeric conversion
   - D) To flush the output stream

   Answer: B) To specify the fill character for padding

8. **Which of the following manipulators will print floating-point numbers in scientific notation?**
   - A) fixed
   - B) showpoint
   - C) scientific
   - D) hex

   Answer: C) scientific

9. **What is the effect of the setprecision manipulator?**
   - A) It sets the number of decimal places in floating-point output.
   - B) It sets the width of the output field.
   - C) It specifies the fill character for padding.
   - D) It sets the base for numeric conversion.

   Answer: A) It sets the number of decimal places in floating-point output.

10. **Which manipulator is used to skip leading whitespace characters in input operations?**
    - A) flush
    - B) setw
    - C) ws
    - D) setfill

    Answer: C) ws

Format State Flags

11. **What does the ios::showbase flag do?**
    - A) It shows the base of the number in the output (e.g., 0x for hexadecimal).
    - B) It sets the precision for floating-point numbers.
    - C) It specifies the fill character for padding.
    - D) It left-justifies the output in a field.

    Answer: A) It shows the base of the number in the output (e.g., 0x for hexadecimal).

12. **Which flag is used to right-justify the output in a field?**
    - A) ios::left
    - B) ios::right
    - C) ios::internal
    - D) ios::showpos

    Answer: B) ios::right

13. **Which format state flag indicates that floating-point numbers should always include a decimal point?**
    - A) ios::fixed
    - B) ios::showpoint
    - C) ios::scientific
    - D) ios::uppercase

    Answer: B) ios::showpoint

14. **What does the ios::internal flag do?**
    - A) Right-justifies numbers in the field.
    - B) Left-justifies numbers in the field.
    - C) Adds padding between the sign and the number.
    - D) Shows the base of the number.

    Answer: C) Adds padding between the sign and the number.

15. **Which function is used to set the field width for input operations?**
    - A) cin.width()
    - B) cin.precision()
    - C) cin.fill()
    - D) cin.setf()

    Answer: A) cin.width()

Additional Stream Manipulations and Manipulators

16. **Which of the following is a valid use of the fixed manipulator?**
    - A) cout << fixed << 123.456;
    - B) cout << fixed << 123;
    - C) cin << fixed << 123.456;
    - D) cin << fixed << 123;

    Answer: A) cout << fixed << 123.456;

17. **What does the setfill manipulator affect when used with setw?**
    - A) It specifies the fill character used for padding the output.
    - B) It sets the number of decimal places for floating-point numbers.

- C) It specifies the width of the output field.
- D) It changes the base for numeric conversion.

Answer: A) It specifies the fill character used for padding the output.

18. **Which manipulator ensures that numbers are printed in uppercase hexadecimal format?**
    - A) hex
    - B) oct
    - C) uppercase
    - D) showbase

    Answer: C) uppercase

19. **How do you set the precision of floating-point output to 3 decimal places?**
    - A) cout << setprecision(3) << 12.34567;
    - B) cout << precision(3) << 12.34567;
    - C) cout << fixed << 12.34567 << setprecision(3);
    - D) cout << 12.34567 << precision(3);

    Answer: A) cout << setprecision(3) << 12.34567;

20. **What is the result of applying `setw(10)` and `setfill('*')` to a number less than 10 characters wide?**
    - A) The number is printed with leading asterisks to make up a total width of 10 characters.
    - B) The number is printed with trailing asterisks to make up a total width of 10 characters.
    - C) The number is printed with trailing spaces to make up a total width of 10 characters.
    - D) The number is printed with leading spaces to make up a total width of 10 characters.

    Answer: A) The number is printed with leading asterisks to make up a total width of 10 characters.

21. **Which of the following manipulators can be used to clear errors from the stream?**
    - A) clear()
    - B) flush()
    - C) ignore()
    - D) reset()

    Answer: A) clear()

22. **What does the `setw` manipulator do when used with an integer value?**

- A) It sets the width of the output field for the integer.
- B) It specifies the number of decimal places for the integer.
- C) It sets the fill character for padding the integer.
- D) It controls the base for integer output.

Answer: A) It sets the width of the output field for the integer.

23. **Which function is used to skip a specified number of characters from the input stream?**
    - A) `cin.ignore()`
    - B) `cin.skip()`
    - C) `cin.flush()`
    - D) `cin.clear()`

    Answer: A) `cin.ignore()`

24. **When using the `hex` manipulator, what will `cout << 255;` output if `hex` is used?**
    - A) 255
    - B) ff
    - C) 0xff
    - D) 0x255

    Answer: B) ff

25. **Which manipulator ensures that all floating-point numbers are printed with the decimal point, even if they are whole numbers?**
    - A) scientific
    - B) fixed
    - C) showpoint
    - D) hex

    Answer: C) showpoint

26. **What is the function of the `flush` manipulator?**
    - A) It removes all input from the input stream.
    - B) It clears the stream's error state.
    - C) It forces the output buffer to be written to the output device.
    - D) It changes the base of numeric output.

    Answer: C) It forces the output buffer to be written to the output device.

27. **Which manipulator is used to control the alignment of the output in a field?**
    - A) `setw()`
    - B) `setfill()`
    - C) left, right, internal
    - D) `setprecision()`

Answer: C) left, right, internal

28. **What is the default state of the ios stream flags in C++ if no manipulators are used?**
    - A) hex
    - B) decimal
    - C) oct
    - D) fixed

    Answer: B) decimal

29. **Which manipulator is used to format output in octal (base-8) representation?**
    - A) hex
    - B) oct
    - C) dec
    - D) base

    Answer: B) oct

30. **What does the showpos manipulator do?**
    - A) It shows the positive sign (+) for positive numbers.
    - B) It shows the base of the number.
    - C) It forces a space before the number.
    - D) It makes numbers output in scientific notation.

    Answer: A) It shows the positive sign (+) for positive numbers.

Lecture #37:

. **Which operator can be overloaded as a non-member function for custom classes?**

- A) +
- B) -
- C) <<
- D) ++

Answer: C) <<

When overloading the stream insertion operator (<<), what must the return type of the function be?

- A) void
- B) ostream&
- C) istream&

- D) int

Answer: B) ostream&

 Which operator is used for stream extraction in C++?

- A) <<
- B) >>
- C) <<|
- D) |>

Answer: B) >>

 When overloading the stream extraction operator (>>), what is mandatory for the second parameter?

- A) It must be passed by value.
- B) It must be passed by reference.
- C) It must be passed as a pointer.
- D) It must be a constant reference.

Answer: B) It must be passed by reference.

 For stream insertion and extraction operators, which parameter represents the stream object?

- A) The first parameter.
- B) The second parameter.
- C) The third parameter.
- D) The return type.

Answer: A) The first parameter.

 What is the return type of the stream extraction operator function?

- A) void
- B) istream&
- C) ostream&
- D) int

Answer: B) istream&

 Which of the following is NOT true about overloading stream insertion (<<) and extraction (>>) operators?

- A) They must be implemented as non-member functions.

- B) They return a reference to the stream object to support cascading operations.
- C) They can be overloaded as member functions of the class.
- D) They take the stream object as the first parameter.

Answer: C) They can be overloaded as member functions of the class.

In the context of overloading stream insertion operator (<<), which of the following is the correct declaration?

- A) `ostream & operator << (Date & d);`
- B) `ostream & operator << (Date d);`
- C) `ostream operator << (ostream & os, Date d);`
- D) `ostream operator << (Date d);`

Answer: B) `ostream & operator << (ostream & os, Date d);`

What does the stream insertion operator (<<) return to support cascading operations?

- A) `void`
- B) `istream&`
- C) `ostream&`
- D) `int`

Answer: C) `ostream&`

Which of the following statements is true about overloading the stream extraction operator (>>)?

- A) It should return `ostream&`.
- B) The second parameter must be passed by reference.
- C) It is commonly used as a member function.
- D) It does not support cascading operations.

Answer: B) The second parameter must be passed by reference.

How is the first parameter of the overloaded stream insertion operator (<<) usually passed?

- A) By value
- B) By pointer
- C) By reference
- D) By constant reference

Answer: C) By reference

 In the overloaded stream insertion operator function, what does the `output` parameter represent?

- A) An `istream` object
- B) An `ostream` object
- C) An `int` object
- D) A `Date` object

Answer: B) An `ostream` object

 For which of the following types of operators is it mandatory to use non-member functions for overloading?

- A) Arithmetic operators
- B) Stream insertion and extraction operators
- C) Unary operators
- D) Comparison operators

Answer: B) Stream insertion and extraction operators

 What is the purpose of the stream insertion operator (<<) in C++?

- A) To input data into a program
- B) To output data from a program
- C) To add two numbers
- D) To subtract two numbers

Answer: B) To output data from a program

 What is the main reason that stream insertion (<<) and extraction (>>) operators cannot be implemented as member functions?

- A) The stream objects (e.g., `cin`, `cout`) are not part of the class.
- B) They need to modify the internal state of the stream.
- C) They require access to private members.
- D) Member functions cannot return references.

Answer: A) The stream objects (e.g., `cin`, `cout`) are not part of the class.

 Which of the following correctly describes the declaration of the overloaded stream extraction operator (>>)?

- A) `istream & operator >> (Date d);`
- B) `istream & operator >> (istream & input, Date & d);`
- C) `istream & operator >> (Date & d);`
- D) `istream operator >> (istream & input, Date d);`

Answer: B) istream & operator >> (istream & input, Date & d);

 What is the first parameter of the overloaded stream extraction operator (>>)?

- A) The object of the class being extracted.
- B) The stream object (e.g., cin).
- C) A constant reference to the stream object.
- D) A temporary object of the class.

Answer: B) The stream object (e.g., cin).

 Which type of function can access private members of a class when overloading the stream insertion and extraction operators?

- A) Member functions
- B) Non-member functions
- C) Static member functions
- D) Friend functions

Answer: D) Friend functions

 What type of operator function is commonly used to overload stream insertion and extraction operators?

- A) Member function
- B) Static member function
- C) Friend function
- D) Global function

Answer: C) Friend function

 What is the purpose of the << operator when used with cout in C++?

- A) To perform mathematical operations.
- B) To output data to the console.
- C) To compare two values.
- D) To perform logical operations.

Answer: B) To output data to the console.

 When overloading the stream extraction operator (>>), why is it necessary to pass the object by reference?

- A) To ensure the object is modified directly.
- B) To avoid modifying the object.
- C) To improve performance.

- D) To allow the function to return a value.

Answer: A) To ensure the object is modified directly.

What should be the return type of the stream extraction operator function to support cascading operations?

- A) void
- B) istream&
- C) ostream&
- D) int

Answer: B) istream&

Which parameter of the stream insertion operator (<<) function is commonly modified within the function?

- A) The stream object (e.g., cout).
- B) The object of the class being inserted.
- C) Both parameters are modified.
- D) None of the parameters are modified.

Answer: B) The object of the class being inserted.

Which of the following statements about stream insertion (<<) and extraction (>>) operators is incorrect?

- A) They must be implemented as non-member functions.
- B) They can be overloaded for custom classes.
- C) They always modify the stream object.
- D) They return a reference to support cascading.

Answer: C) They always modify the stream object.

Which operator is used to input data from the standard input stream in C++?

- A) <<
- B) >>
- C) <<|
- D) |>

Answer: B) >>

What type of access does a friend function have to the class in which it is declared?

- A) Public access

- B) Protected access
- C) Private access
- D) No special access

Answer: C) Private access

 Which parameter of the overloaded stream insertion operator (<<) is a reference to the stream object?

- A) The second parameter
- B) The first parameter
- C) Both parameters
- D) None of the parameters

Answer: B) The first parameter

 What is a common use case for overloading stream insertion and extraction operators?

- A) Performing arithmetic calculations
- B) Formatting output for custom classes
- C) Comparing objects
- D) Modifying object states

Answer: B) Formatting output for custom classes

 What does the << operator do when used with cout in a cascading statement like cout << i << d << f;?

- A) It performs multiple operations in one line.
- B) It outputs all values in a single line.
- C) It compares the values of i, d, and f.
- D) It formats the values of i, d, and f for input.

Answer: B) It outputs all values in a single line.

 What is the main advantage of overloading the stream insertion (<<) and extraction (>>) operators for custom classes?

- A) It allows performing arithmetic operations on custom class objects.
- B) It provides a convenient and readable way to input and output custom class data.
- C) It enhances the performance of custom class operations.
- D) It automatically converts custom class objects to native types.

Answer: B) It provides a convenient and readable way to input and output custom class

data.

Lecture # 38

. User-Defined Manipulators

1. **What is a user-defined manipulator in C++?**
   - A) A function that manipulates data types
   - B) A function that manipulates the output stream formatting
   - C) A function that alters the internal state of an object
   - D) A function that handles memory management

   Answer: B) A function that manipulates the output stream formatting

2. **How should a user-defined manipulator function be declared?**
   - A) ostream manipulator_name(ostream& os)
   - B) ostream& manipulator_name(ostream& os)
   - C) ostream manipulator_name()
   - D) ostream& manipulator_name()

   Answer: B) ostream& manipulator_name(ostream& os)

3. **What does the following user-defined manipulator function do?**

   cpp
   Copy code
   ```
   ostream& bell(ostream& os) {
      return os << '\a';
   }
   ```

   - A) Inserts a newline character
   - B) Inserts a tab character
   - C) Inserts a bell character
   - D) Flushes the output buffer

   Answer: C) Inserts a bell character

4. **Given the following code snippet, what will be the output?**

   cpp
   Copy code
   ```
   ostream& tab(ostream& os) {
   ```

```
    return os << '\t';
}

int main() {
    cout << "Hello" << tab << "World";
    return 0;
}
```

- A) HelloWorld
- B) Hello\tWorld
- C) Hello World
- D) HelloWorld\t

Answer: C) Hello World

5. **What is the purpose of using `endl` in the output stream?**
   - A) To insert a tab character
   - B) To insert a newline character and flush the buffer
   - C) To insert a carriage return
   - D) To insert a bell character

   Answer: B) To insert a newline character and flush the buffer

6. **Which of the following is a correct way to define a user-defined manipulator that sets the output to uppercase?**
   - A)

     ```
     cpp
     Copy code
     ostream& uppercase(ostream& os) {
         os.setf(ios::uppercase);
         return os;
     }
     ```

   - B)

     ```
     cpp
     Copy code
     ostream uppercase(ostream& os) {
         os.setf(ios::uppercase);
         return os;
     }
     ```

   - C)

     ```
     cpp
     Copy code
     ostream& uppercase() {
         return cout.setf(ios::uppercase);
     }
     ```

- D)

```
cpp
Copy code
ostream uppercase() {
   return cout.setf(ios::uppercase);
}
```

7. **Answer:** A)

```
cpp
Copy code
ostream& uppercase(ostream& os) {
   os.setf(ios::uppercase);
   return os;
}
```

14. **What is the output of the following code?**

```
cpp
Copy code
ostream& hexFormat(ostream& os) {
   return os << hex;
}

int main() {
   int num = 255;
   cout << hexFormat << num;
   return 0;
}
```

- A) 255
- B) 0xff
- C) ff
- D) 255

Answer: C) ff

15. **What will be the output of the following code?**

```
cpp
Copy code
ostream& reset(ostream& os) {
   os.unsetf(ios::hex);
   return os;
}

int main() {
   cout << hex << 255 << reset << 255;
   return 0;
}
```

- A) 0xff255
- B) ff255
- C) 255255
- D) 0xff 255

Answer: C) 255255

16. **How can you use a user-defined manipulator to set the precision of floating-point output?**
    - A) By defining a manipulator that sets the precision using ios::precision
    - B) By defining a manipulator that uses cout.precision()
    - C) By defining a manipulator that uses cout.setf()
    - D) By defining a manipulator that uses cout.width()

    Answer: A) By defining a manipulator that sets the precision using ios::precision

17. **What does the following user-defined manipulator do?**

```
cpp
Copy code
ostream& flushBuffer(ostream& os) {
   os.flush();
   return os;
}
```

    - A) Clears the output buffer
    - B) Inserts a newline character and flushes the buffer
    - C) Flushes the output buffer
    - D) Inserts a tab character

    Answer: C) Flushes the output buffer

Static Keyword

11. **What does the static keyword do when used with a local variable inside a function?**
    - A) The variable is only visible within the function but retains its value between function calls.
    - B) The variable is visible throughout the entire program and retains its value.
    - C) The variable is visible within the function and does not retain its value.
    - D) The variable is initialized every time the function is called.

    Answer: A) The variable is only visible within the function but retains its value between function calls.

12. **What will happen if you declare a static variable inside a function without**

initializing it?

- A) The variable will be initialized to zero automatically.
- B) The variable will be uninitialized and may contain garbage values.
- C) The program will fail to compile.
- D) The variable will be initialized to a random value.

Answer: A) The variable will be initialized to zero automatically.

13. **Which of the following is true about static data members of a class?**
    - A) They are initialized once per object of the class.
    - B) They are shared among all objects of the class.
    - C) They are initialized every time an object is created.
    - D) They exist only during the lifetime of an object.

    Answer: B) They are shared among all objects of the class.

14. **How do you initialize a static data member of a class?**
    - A) Inside the class definition
    - B) In the constructor of the class
    - C) Outside the class definition, using the scope resolution operator
    - D) In the destructor of the class

    Answer: C) Outside the class definition, using the scope resolution operator

15. **What is the primary advantage of using static variables in a class?**
    - A) They provide a unique value for each object.
    - B) They maintain their value between function calls.
    - C) They allow for sharing common data among all objects of the class.
    - D) They are automatically initialized to a specific value.

    Answer: C) They allow for sharing common data among all objects of the class.

16. **What does the `static` keyword do when used with a member function of a class?**
    - A) The member function can only be called by the class itself.
    - B) The member function does not have access to the `this` pointer.
    - C) The member function can only be called by objects of the class.
    - D) The member function can only be called within the scope of the member variable.

    Answer: B) The member function does not have access to the `this` pointer.

17. **What is the output of the following code?**

```
cpp
Copy code
class Example {
public:
```

```
    static int value;
    void increment() { ++value; }
};

int Example::value = 0;

int main() {
    Example ex1, ex2;
    ex1.increment();
    ex2.increment();
    cout << Example::value;
    return 0;
}
```

- A) 0
- B) 1
- C) 2
- D) 3

Answer: C) 2

18. **What will be the output if a static variable is declared in a function and that function is called multiple times?**
    - A) The static variable is reinitialized to zero each time the function is called.
    - B) The static variable retains its value across multiple function calls.
    - C) The function will not compile if a static variable is used.
    - D) The static variable is unique to each function call.

    Answer: B) The static variable retains its value across multiple function calls.

19. **In which of the following scenarios is the `static` keyword used?**
    - A) To make a function local to the file
    - B) To make a variable or function accessible across multiple files
    - C) To make a member function of a class operate on class-level data
    - D) To make a class member unique to each object

    Answer: C) To make a member function of a class operate on class-level data

20. **What does the following code print?**

```
cpp
Copy code
class Counter {
public:
    static int count;
    Counter() { ++count; }
};

int Counter::count = 0;
```

```
int main() {
   Counter c1, c2, c3;
   cout << Counter::count;
   return 0;
}
```

- A) 0
- B) 1
- C) 2
- D) 3

Answer: D) 3

Mixed Questions

21. **Which of the following is a correct syntax to declare a static member variable in a class?**
    - A) `static int variable;`
    - B) `int static variable;`
    - C) `static variable int;`
    - D) `int variable static;`

    Answer: A) `static int variable;`

22. **What is the effect of using `static` keyword with a global variable?**
    - A) The variable is visible only within the file where it is declared.
    - B) The variable is visible throughout the entire program.
    - C) The variable is visible only within the function where it is declared.
    - D) The variable is only visible in the class where it is declared.

    Answer: A) The variable is visible only within the file where it is declared.

23. **What is the purpose of the `static` keyword in a function definition?**
    - A) To limit the scope of the function
    - B) To retain the value of local variables between function calls
    - C) To make the function accessible only within the class
    - D) To initialize variables with a fixed value

    Answer: B) To retain the value of local variables between function calls

24. **Which of the following will not change the behavior of the `endl` manipulator?**
    - A) Flushing the output buffer
    - B) Inserting a newline character
    - C) Inserting a tab character
    - D) Ensuring text output appears immediately

Answer: C) Inserting a tab character

25. **What will the following code output?**

```
cpp
Copy code
class Test {
public:
   static int data;
   Test() { data++; }
};

int Test::data = 10;

int main() {
   Test t1;
   Test t2;
   cout << Test::data;
   return 0;
}
```

    - A) 10
    - B) 11
    - C) 12
    - D) 20

    Answer: B) 12

26. **Which of the following is the correct way to define a static member function in a class?**
    - A) `static void func();`
    - B) `void static func();`
    - C) `void func() static;`
    - D) `static func void();`

    Answer: A) `static void func();`

27. **What is the purpose of using the `static` keyword in class member functions?**
    - A) To create an instance-specific function
    - B) To restrict the function's access to specific objects
    - C) To allow the function to be called without an object
    - D) To make the function private to the class

    Answer: C) To allow the function to be called without an object

28. **What will the following code print?**

```
cpp
Copy code
```

```
class Example {
public:
   static int x;
   static void setX(int value) { x = value; }
};

int Example::x = 0;

int main() {
   Example::setX(100);
   cout << Example::x;
   return 0;
}
```

- A) 0
- B) 10
- C) 100
- D) 1000

Answer: C) 100

29. **What will be the output of the following code?**

```
cpp
Copy code
class MyClass {
public:
   static int value;
   void setValue(int val) { value = val; }
};

int MyClass::value = 50;

int main() {
   MyClass obj;
   obj.setValue(150);
   cout << MyClass::value;
   return 0;
}
```

- A) 50
- B) 150
- C) 100
- D) 200

Answer: B) 150

30. **In the context of C++ programming, what is the effect of the `static` keyword when used with an object in a class?**
    - A) It ensures that each object gets a unique copy of the member variable.
    - B) It makes the member variable accessible only through static member

functions.
- C) It makes the member variable shared among all instances of the class.
- D) It restricts the member variable to be modified only through constructors.

Answer: C) It makes the member variable shared among all instances of the class.

Lecture # 39

. 1. What is the main purpose of a pointer in C++?

a) To store a value directly
b) To store the address of another variable
c) To store a reference to a variable
d) To create a new variable

Answer: b) To store the address of another variable

2. Which operator is used to obtain the address of a variable in C++?

a) *
b) &
c) ->
d) ++

Answer: b) &

3. What does a reference in C++ represent?

a) A type of pointer that can be null
b) An alias for another variable
c) A special type of pointer that can be reassigned
d) A pointer to a constant

Answer: b) An alias for another variable

4. What is the result of using the new operator in C++?

a) It deallocates memory from the free store
b) It allocates memory from the free store
c) It initializes a variable

d) It creates a static variable

Answer: b) It allocates memory from the free store

5. How do you deallocate memory allocated with the `new` operator?

a) delete (pointer)
b) delete pointer
c) free (pointer)
d) dealloc (pointer)

Answer: a) delete (pointer)

6. What is the correct syntax to allocate memory for an array of 10 integers using the `new` operator?

a) int *p = new int(10);
b) int *p = new int[10];
c) int p = new int[10];
d) int p = new int(10);

Answer: b) int *p = new int[10];

7. What does the `delete []` operator do in C++?

a) It frees the memory allocated for a single variable
b) It initializes an array
c) It frees the memory allocated for an array
d) It allocates additional memory for an array

Answer: c) It frees the memory allocated for an array

8. What is a copy constructor used for in C++?

a) To initialize an object using another object of the same class
b) To assign one object to another
c) To delete an object
d) To allocate dynamic memory

Answer: a) To initialize an object using another object of the same class

9. What does a default copy constructor do?

a) Performs a deep copy of the object
b) Performs a shallow copy of the object
c) Deletes the original object

d) Initializes the object with default values

Answer: b) Performs a shallow copy of the object

10. What is the primary purpose of defining a destructor in a class?

a) To allocate memory
b) To perform a shallow copy
c) To deallocate memory
d) To create a copy of the object

Answer: c) To deallocate memory

11. Which of the following statements about pointers is true?

a) A pointer can be assigned to another pointer of a different type without casting.
b) A pointer can be incremented or decremented to access different elements in an array.
c) A pointer always points to a constant value.
d) A pointer must always be initialized with a non-null value.

Answer: b) A pointer can be incremented or decremented to access different elements in an array.

12. What is the result of using the following code: `Matrix m2 = m1;` if the Matrix class has a default copy constructor?

a) m2 will be an independent copy of m1.
b) m2 will point to the same memory as m1.
c) m2 will have the same values but a different memory allocation.
d) m2 will not be properly initialized.

Answer: b) m2 will point to the same memory as m1.

13. What should you check for in an assignment operator to prevent self-assignment?

a) Whether the addresses of the objects are the same
b) Whether the values of the objects are the same
c) Whether the objects are of the same type
d) Whether the objects are in the same scope

Answer: a) Whether the addresses of the objects are the same

14. Which keyword is used to define a constant reference in a function

parameter list?

a) static
b) const
c) volatile
d) auto

Answer: b) const

15. In a class that manages dynamic memory, what should be done in the destructor?

a) Allocate additional memory
b) Copy the current memory to a new location
c) Deallocate the memory previously allocated
d) Initialize the memory to default values

Answer: c) Deallocate the memory previously allocated

Feel free to use or modify these questions according to your needs!

why u stop i ask for 30 mcqs

ChatGPT

Of course! Here are 15 additional multiple-choice questions to make a total of 30 questions on pointers, references, dynamic memory allocation, and copy constructors in C++:

16. What is the result of dereferencing a null pointer?

a) It causes a compiler error
b) It returns a default value
c) It causes undefined behavior
d) It results in a syntax error

Answer: c) It causes undefined behavior

17. Which of the following is a correct way to declare a pointer to a constant integer?

a) const int* ptr;
b) int* const ptr;
c) const int const* ptr;
d) int const* ptr;

Answer: a) const int* ptr;

18. What does the -> operator do in C++?

a) It dereferences a pointer and accesses a member of the pointed-to object
b) It increments the value of the pointer
c) It deallocates memory
d) It creates a new pointer

Answer: a) It dereferences a pointer and accesses a member of the pointed-to object

19. How can you initialize a pointer to an integer variable named x?

a) int *p = x;
b) int *p = &x;
c) int p = &x;
d) int p = x;

Answer: b) int *p = &x;

20. What will happen if you forget to delete memory allocated with new?

a) The program will execute without errors
b) The memory will be automatically managed by the compiler
c) The memory will remain allocated and cause a memory leak
d) The memory will be deallocated when the program ends

Answer: c) The memory will remain allocated and cause a memory leak

21. Which of the following statements is true about a pointer to a pointer?

a) It points to the address of another pointer
b) It points to the address of a variable
c) It points to the value of another pointer
d) It cannot be used to dynamically allocate memory

Answer: a) It points to the address of another pointer

22. What is a deep copy?

a) Copying the pointer's value
b) Copying the values pointed to by the pointer
c) Copying the address of the pointer
d) Copying only the metadata of the object

Answer: b) Copying the values pointed to by the pointer

23. What keyword is used to declare a pointer that cannot be reassigned after initialization?

a) static
b) volatile
c) const
d) final

Answer: c) const

24. How can you safely copy a dynamically allocated array?

a) Use the default copy constructor
b) Perform a deep copy by allocating new memory and copying each element individually
c) Use `memcpy` for copying the memory
d) Use `std::copy` to copy the array

Answer: b) Perform a deep copy by allocating new memory and copying each element individually

25. Which of the following is true about references in C++?

a) A reference must be initialized when declared
b) A reference can be null
c) A reference can be reassigned to refer to another variable
d) A reference can be initialized without an existing variable

Answer: a) A reference must be initialized when declared

26. What is the primary difference between `new` and `malloc`?

a) `new` initializes objects while `malloc` does not
b) `malloc` initializes objects while `new` does not
c) `new` is used for allocating arrays, while `malloc` is not
d) `malloc` throws exceptions on failure, while `new` does not

Answer: a) `new` initializes objects while `malloc` does not

27. What is the purpose of the `const` keyword in a function parameter list when using pointers?

a) To specify that the pointer itself cannot be modified
b) To specify that the value pointed to by the pointer cannot be modified
c) To specify that the pointer will point to a constant variable

d) To indicate that the function will not modify the pointer

Answer: b) To specify that the value pointed to by the pointer cannot be modified

28. What is the result of using the following code: `delete p; p = nullptr;`?

a) The memory is deallocated, and the pointer `p` remains pointing to the old memory
b) The memory is deallocated, and the pointer `p` is set to null
c) The memory is not deallocated, and the pointer `p` is set to null
d) The memory is deallocated, but setting `p` to null has no effect

Answer: b) The memory is deallocated, and the pointer `p` is set to null

29. What will happen if you attempt to delete an already-deleted pointer?

a) The pointer will be reallocated
b) The program will throw an exception
c) The program may crash or cause undefined behavior
d) The pointer will be automatically reset

Answer: c) The program may crash or cause undefined behavior

30. What is the correct way to declare a copy constructor in a C++ class?

a) `ClassName(const ClassName &obj);`
b) `ClassName(ClassName obj);`
c) `ClassName(ClassName &obj);`
d) `ClassName(const ClassName obj);`

Answer: a) `ClassName(const ClassName &obj);`

Lecture # 40:

. 1. When a class contains objects of other classes as data members, which of the following statements is true?

a) The contained objects are constructed after the containing object.
b) The containing object is constructed before the contained objects.
c) The construction order is arbitrary.
d) The destruction order is the same as the construction order.

Answer: b) The containing object is constructed before the contained objects.

2. In the context of object construction and destruction, what happens to the contained objects?

a) They are destroyed before the containing object.
b) They are constructed after the containing object.
c) They are constructed and destroyed in the same order as the containing object.
d) They are constructed and destroyed at the same time as the containing object.

Answer: a) They are destroyed before the containing object.

3. What is the purpose of an initializer list in C++?

a) To initialize pointers to objects.
b) To initialize contained objects during the construction of a containing object.
c) To allocate dynamic memory.
d) To perform runtime checks.

Answer: b) To initialize contained objects during the construction of a containing object.

4. What is the default visibility of members in a `struct` in C++?

a) Public
b) Protected
c) Private
d) None of the above

Answer: a) Public

5. What is the default visibility of members in a `class` in C++?

a) Public
b) Protected
c) Private
d) None of the above

Answer: c) Private

6. How do nested classes in C++ differ from regular classes?

a) They are visible outside the enclosing class regardless of their access specifier.
b) They are visible only within the enclosing class.
c) They can have constructors and destructors but no access to the enclosing class's private members.
d) They are always considered friends of the enclosing class.

Answer: b) They are visible only within the enclosing class.

7. What keyword is used to declare a nested class in C++?

a) nested
b) private
c) public
d) No special keyword is needed.

Answer: d) No special keyword is needed.

8. When an outer class is destroyed, what happens to the nested classes?

a) They are destroyed before the outer class.
b) They are destroyed after the outer class.
c) They are not affected.
d) Their destruction is independent of the outer class.

Answer: b) They are destroyed after the outer class.

9. What happens if a nested class is declared in the private section of an outer class?

a) It is visible to all classes.
b) It is visible only to the outer class.
c) It is not visible to the outer class.
d) It can be accessed only by the outer class and its friends.

Answer: b) It is visible only to the outer class.

10. What can a nested class access within its enclosing class?

a) Only the public members of the enclosing class.
b) Only the private members of the enclosing class.
c) All members of the enclosing class.
d) No members of the enclosing class.

Answer: c) All members of the enclosing class.

11. In the provided example, what is the order of constructor and destructor calls?

a) B's constructor, A's constructor, A's destructor, B's destructor
b) A's constructor, B's constructor, B's destructor, A's destructor
c) B's constructor, A's constructor, B's destructor, A's destructor
d) A's constructor, B's constructor, A's destructor, B's destructor

Answer: b) A's constructor, B's constructor, B's destructor, A's destructor

12. In the code example provided, what will be the output when an object of class `B` is created?

a) A Constructor ... B Constructor ... B Destructor ... A Destructor ...
b) B Constructor ... A Constructor ... A Destructor ... B Destructor ...
c) A Constructor ... B Constructor ... A Destructor ... B Destructor ...
d) B Constructor ... A Constructor ... B Destructor ... A Destructor ...

Answer: b) B Constructor ... A Constructor ... A Destructor ... B Destructor ...

13. If a nested class is declared as `public`, what does this mean for its visibility?

a) It is visible only within the enclosing class.
b) It is visible outside the enclosing class.
c) It is accessible only to the members of the enclosing class.
d) It is accessible only to the outer class's friends.

Answer: b) It is visible outside the enclosing class.

14. What access does a nested class have to the members of the enclosing class if it is private?

a) It can access only public members of the enclosing class.
b) It can access only private members of the enclosing class.
c) It has no special access to members of the enclosing class.
d) It can access all members, including private ones, of the enclosing class.

Answer: b) It can access only private members of the enclosing class.

15. What happens to the memory allocated for a nested class object?

a) It is automatically deallocated when the outer class is destroyed.
b) It needs to be explicitly deallocated.
c) It is not allocated memory by default.
d) It is managed independently of the outer class.

Answer: a) It is automatically deallocated when the outer class is destroyed.

16. Can a nested class be a friend of the outer class?

a) Yes, using the `friend` keyword.
b) No, nested classes cannot be friends of outer classes.
c) Yes, but only if the outer class is a friend of the nested class.

d) No, it is not allowed in C++.

Answer: a) Yes, using the `friend` keyword.

17. What is the purpose of nesting classes within other classes?

a) To allow classes to share their implementation details with one another.
b) To avoid using inheritance.
c) To ensure that all classes have a common constructor.
d) To prevent multiple inheritance.

Answer: a) To allow classes to share their implementation details with one another.

18. What happens to the visibility of a nested class if it is declared in the private section of the outer class?

a) It is visible only to the outer class and its friends.
b) It is visible to all classes in the program.
c) It is visible to all nested classes of the outer class.
d) It is accessible only through static methods of the outer class.

Answer: a) It is visible only to the outer class and its friends.

19. How can you access a private member of an outer class from a nested class?

a) Directly, if the nested class is defined within the private section of the outer class.
b) Using a public method of the outer class.
c) By declaring the nested class as a friend of the outer class.
d) By making the nested class public.

Answer: a) Directly, if the nested class is defined within the private section of the outer class.

20. What should be used to initialize contained objects in a class?

a) Constructor initialization list
b) Destructor
c) Member functions
d) Static methods

Answer: a) Constructor initialization list

21. Which of the following is true about structures used as class members?

a) They can be initialized with constructors.
b) They cannot be used as members in a class.
c) They are automatically initialized with default values.
d) They have different destruction behavior compared to classes.

Answer: a) They can be initialized with constructors.

22. What is the main advantage of using classes as members of other classes?

a) They allow for modular and organized code.
b) They reduce code redundancy.
c) They simplify memory management.
d) They enhance the performance of the code.

Answer: a) They allow for modular and organized code.

23. In C++, how is memory for a nested class managed?

a) It is managed by the outer class's memory management.
b) It is managed independently of the outer class.
c) It is managed by the operating system.
d) It is managed by the compiler's memory management system.

Answer: a) It is managed by the outer class's memory management.

24. What is the effect of declaring a nested class in the `public` section of the outer class?

a) The nested class is accessible outside the outer class.
b) The nested class is not accessible from outside the outer class.
c) The nested class is accessible only to the outer class.
d) The nested class is accessible only to the members of the outer class.

Answer: a) The nested class is accessible outside the outer class.

25. How does C++ handle the destruction of nested classes?

a) Nested classes are destroyed after the outer class.
b) Nested classes are destroyed before the outer class.
c) Nested classes are not automatically destroyed; they must be manually deallocated.
d) Nested classes are destroyed independently of the outer class.

Answer: b) Nested classes are destroyed before the outer class.

26. What happens if a member function of a nested class is called?

a) The function can directly access the members of the enclosing class.
b) The function requires a pointer to the enclosing class.
c) The function cannot access any members of the enclosing class.
d) The function can only access static members of the enclosing class.

Answer: a) The function can directly access the members of the enclosing class.

27. Can you define a class inside a `struct` in C++?

a) Yes, C++ allows defining classes inside structs.
b) No, classes cannot be defined inside structs.
c) Yes, but only if the class is public.
d) No, structs are only for data members and cannot contain classes.

Answer: a) Yes, C++ allows defining classes inside structs.

28. What happens to the contained object when an instance of a class containing that object is created?

a) The contained object is constructed first, followed by the containing object.
b) The containing object is constructed first, followed by the contained object.
c) Both objects are constructed simultaneously.
d) The contained object is constructed only if explicitly initialized.

Answer: b) The containing object is constructed first, followed by the contained object.

29. Can a nested class access the private members of its enclosing class?

a) Yes, if the nested class is declared within the private section of the enclosing class.
b) No, nested classes cannot access private members.
c) Yes, but only if the nested class is declared as a friend.
d) No, nested classes can only access public members.

Answer: a) Yes, if the nested class is declared within the private section of the enclosing class.

30. How do you initialize contained objects in a class when using an initializer list?

a) By directly setting their values in the constructor's body.
b) By passing their values as arguments in the initializer list of the containing class's constructor.
c) By calling their constructors after the containing class's constructor has run.

d) By using static methods of the contained class.

Answer: b) By passing their values as arguments in the initializer list of the containing class's constructor.

Lecture#41:

. What is the main purpose of function templates in C++?

a) To allow functions to operate with different data types using a single definition.
b) To overload functions based on their return type.
c) To enable functions to be defined with multiple return types.
d) To allow functions to operate only with built-in data types.

Answer: a) To allow functions to operate with different data types using a single definition.

Which keyword is used to define a template in C++?

a) template
b) generic
c) type
d) class

Answer: a) template

What does the following function template definition do?

```
cpp
Copy code
template<class T>
T reverse(T x)
{
   return (-x);
}
```

a) It returns the negative value of the input.
b) It returns the absolute value of the input.
c) It multiplies the input by -1.
d) It reverses the digits of the input number.

Answer: a) It returns the negative value of the input.

What happens when you call a template function with a data type that was not

previously specified?

a) The compiler generates a specific version of the function for that data type.
b) The program will not compile.
c) The function will fail to execute at runtime.
d) The compiler will use a default implementation of the function.

Answer: a) The compiler generates a specific version of the function for that data type.

 Given the following template function:

```
cpp
Copy code
template<class T>
void swap(T &x, T &y)
{
   T tmp;
   tmp = x;
   x = y;
   y = tmp;
}
```

What will happen if `swap` is called with `int` and `double` types?

a) The code will compile and run, but with unpredictable results.
b) The compiler will generate a swap function for each data type separately.
c) The code will not compile because both arguments must be of the same type.
d) The compiler will automatically cast the `int` to `double`.

Answer: c) The code will not compile because both arguments must be of the same type.

 How does the compiler handle function templates when they are used with different data types?

a) It creates a separate version of the function for each data type used.
b) It uses the same function definition for all data types.
c) It throws a compile-time error.
d) It generates runtime errors.

Answer: a) It creates a separate version of the function for each data type used.

 What is the result of calling the function `larger` with mixed data types, such as `int` and `double`, when `larger` is defined as follows?

```
cpp
Copy code
template<class T>
```

```
T larger(T x, T y)
{
   T big;
   if (x > y)
      big = x;
   else
      big = y;
   return big;
}
```

a) The code will compile and run with type casting.
b) The compiler will generate a function for each data type used.
c) The code will not compile due to type mismatch.
d) The compiler will use the int version of the function.

Answer: c) The code will not compile due to type mismatch.

When defining a template function with multiple template parameters, what is the syntax?

a) `template<class T, class U>`
b) `template<class T> class U`
c) `template<type T, type U>`
d) `template<T, U>`

Answer: a) `template<class T, class U>`

Which of the following statements about template functions is false?

a) Templates can only be used with functions.
b) Template functions can have multiple generic parameters.
c) The template definition must be in the same file as its usage.
d) Templates are instantiated at compile time.

Answer: a) Templates can only be used with functions.

What does the following template function do?

```
cpp
Copy code
template<class T>
void print(T value)
{
   std::cout << value << std::endl;
}
```

a) Prints the value to the console for any data type.
b) Returns the value for any data type.
c) Takes input and prints it to the console only for integers.

d) Formats the value and prints it.

Answer: a) Prints the value to the console for any data type.

 Can you use function templates with class member functions in C++?

a) Yes, as long as the member functions are public.
b) No, function templates can only be used with non-member functions.
c) Yes, but only if the member functions are static.
d) No, you need to use non-template member functions.

Answer: a) Yes, as long as the member functions are public.

 What will be the result if you use `template<class T> T reverse(T x)` with a class that does not have an overloaded - operator?

a) The compiler will generate a default implementation of the `-` operator.
b) The code will compile and run, but `reverse` will not work correctly.
c) The compiler will throw a compile-time error.
d) The code will compile, but will cause a runtime error.

Answer: c) The compiler will throw a compile-time error.

 How does explicit template specialization work?

a) It allows the definition of a specific template for a particular data type.
b) It requires manual implementation for every data type.
c) It replaces all instances of the template with a specific data type at runtime.
d) It generates code based on a default data type.

Answer: a) It allows the definition of a specific template for a particular data type.

 Which of the following is a correct way to explicitly specify a template type in C++?

a) `reverse<int>(a);`
b) `reverse<type>(a);`
c) `reverse<template<int>>(a);`
d) `reverse<int>::method(a);`

Answer: a) `reverse<int>(a);`

 What does the following code snippet do?

cpp
Copy code
```
template<class T>
void printArray(T arr[], int size)
```

```
{
   for (int i = 0; i < size; i++)
      std::cout << arr[i] << " ";
}
```

a) Prints each element of the array to the console.
b) Returns the array elements as a string.
c) Sorts the array in ascending order.
d) Modifies each element of the array.

Answer: a) Prints each element of the array to the console.

 In which scenario is using function templates preferred over function overloading?

a) When the same logic applies to multiple data types and needs to be reused.
b) When functions have different logic but similar names.
c) When you want to enforce strict type checking.
d) When dealing with runtime polymorphism.

Answer: a) When the same logic applies to multiple data types and needs to be reused.

 What is a limitation of function templates in C++?

a) Templates cannot be used with class member functions.
b) Templates must be defined and declared in the same file.
c) Templates cannot have multiple parameters.
d) Templates require explicit type definition at runtime.

Answer: b) Templates must be defined and declared in the same file.

 How do you handle a situation where you want to define a template that works with both integers and floating-point numbers?

a) Use template specialization for each type.
b) Define separate functions for each data type.
c) Use a single template function with generic type parameters.
d) Define a macro for each data type.

Answer: c) Use a single template function with generic type parameters.

 What is the output of the following code if `reverse` is a template function as shown?

```
cpp
Copy code
template<class T>
T reverse(T x)
{
   return -x;
```

```
}

int main()
{
   double a = 5.5;
   std::cout << reverse(a) << std::endl;
   return 0;
}
```

a) -5.5
b) 5.5
c) 0
d) 5

Answer: a) -5.5

What will happen if you use a template function with a data type that does not support the operations used in the function?

a) The function will not compile if the operations are not supported.
b) The function will compile, but will result in runtime errors.
c) The compiler will automatically provide default implementations.
d) The program will compile and run with the closest matching data type.

Answer: a) The function will not compile if the operations are not supported.

What is the role of the typename keyword in template definitions?

a) It specifies that the template parameter is a type.
b) It is used to define the function body.
c) It replaces the class keyword in template definitions.
d) It indicates that the parameter is a constant value.

Answer: a) It specifies that the template parameter is a type.

Which of the following is a valid way to define a class template in C++?

a) template<class T> class MyClass { T data; };
b) class template<T> MyClass { T data; };
c) template<MyClass<T>> class T { T data; };
d) class template<T> MyClass { T data; };

Answer: a) template<class T> class MyClass { T data; };

What is the result of the following code if the template function is used?

cpp
Copy code

```
template<typename T>
void display(T value)
{
   std::cout << "Value: " << value << std::endl;
}

int main()
{
   display(10);
   display("Hello");
   return 0;
}
```

a) The code will compile and print Value: 10 and Value: Hello.
b) The code will compile and run but produce a compilation error.
c) The code will print Value: 10 and Value: 0.
d) The code will not compile because of the display function.

Answer: a) The code will compile and print Value: 10 and Value: Hello.

 What does it mean when a function template is instantiated?

a) The compiler generates a concrete version of the function for the specific type.
b) The function is converted to a non-template function.
c) The function is optimized at runtime.
d) The function is replaced with default implementations.

Answer: a) The compiler generates a concrete version of the function for the specific type.

 Which of the following statements about class templates is true?

a) Class templates can only have one template parameter.
b) Class templates can have multiple template parameters.
c) Class templates must have a specific type defined at compile-time.
d) Class templates cannot be used with inheritance.

Answer: b) Class templates can have multiple template parameters.

 How does the compiler determine which version of a template function to use?

a) By matching the data types of the function arguments with the template parameters.
b) By selecting the most recently defined version of the function.
c) By analyzing the runtime values of the arguments.
d) By checking the order of function definitions.

Answer: a) By matching the data types of the function arguments with the template parameters.

 What is the purpose of `template<class T>` in the following code?

```cpp
Copy code
template<class T>
class Stack {
private:
   T* arr;
   int top;
   int capacity;
public:
   Stack(int size);
   void push(T element);
   T pop();
   bool isEmpty();
};
```

a) To declare a class that can work with any data type.
b) To define a class with a specific data type.
c) To specify the return type of the class methods.
d) To create a stack that only accepts `int` data types.

Answer: a) To declare a class that can work with any data type.

 What will the following code output if `swap` is a function template?

```cpp
Copy code
template<class T>
void swap(T &a, T &b)
{
   T temp = a;
   a = b;
   b = temp;
}

int main()
{
   int x = 1, y = 2;
   swap(x, y);
   std::cout << x << " " << y << std::endl;
   return 0;
}
```

a) 2 1
b) 1 2
c) 1 1
d) 2 2

Answer: a) 2 1

 What happens when you use a template with an invalid data type?

a) The compiler generates a compile-time error.
b) The template function will execute but return incorrect results.
c) The program will compile, but the template function will not be executed.
d) The template function will automatically cast the data type.

Answer: a) The compiler generates a compile-time error.

 In which scenario should you prefer using template specialization over regular templates?

a) When you need a different implementation of a template for specific types.
b) When the template needs to be used for multiple data types with the same logic.
c) When you want to handle generic types without special cases.
d) When implementing generic algorithms with no specific requirements.

Answer: a) When you need a different implementation of a template for specific types.

Lecture # 42:

. What is the main purpose of using class templates in C++?

- A) To define a class that can work with different data types
- B) To create multiple versions of a class for different data types
- C) To provide a mechanism for code duplication
- D) To enhance runtime efficiency by optimizing specific data types

Answer: A) To define a class that can work with different data types

2. How do you declare a template class with a non-type parameter?

- A) template <class T, int N> class MyClass;
- B) template <class T> class MyClass<int N>;
- C) template <class T, typename N> class MyClass;
- D) template <int N, class T> class MyClass;

Answer: A) template <class T, int N> class MyClass;

3. In a template class, what is the behavior of static member variables?

- A) Static member variables are unique to each instance of the class.
- B) Static member variables are shared across all instances of the class, regardless of the template parameters.

- C) Static member variables are private to the template class and cannot be accessed from outside.
- D) Static member variables are recreated for each different template instantiation.

Answer: B) Static member variables are shared across all instances of the class, regardless of the template parameters.

4. What is a correct syntax to define a member function outside the class in a template class?

- A) template <class T> void MyClass<T>::memberFunction() { // definition }
- B) template <T> void MyClass::memberFunction() { // definition }
- C) template <class T> void memberFunction() { // definition }
- D) template <class T> void MyClass::memberFunction(T arg) { // definition }

Answer: A) template <class T> void MyClass<T>::memberFunction() { // definition }

5. How do friend functions behave in template classes?

- A) Friend functions are only accessible to the specific instantiation of the template class for which they are declared.
- B) Friend functions have access to private members of all instantiations of the template class.
- C) Friend functions cannot be used with template classes.
- D) Friend functions are only accessible to member functions of the template class.

Answer: A) Friend functions are only accessible to the specific instantiation of the template class for which they are declared.

6. What is the advantage of using template classes over multiple class definitions for different data types?

- A) Templates require less memory.
- B) Templates automatically manage memory allocation.
- C) Templates avoid code duplication and promote code reuse.
- D) Templates improve runtime performance.

Answer: C) Templates avoid code duplication and promote code reuse.

7. When a template class is instantiated with a non-type parameter, how is the parameter used in the class?

- A) As a constant value within the class

- B) As a template parameter that determines the type of the class
- C) To define the size of arrays within the class
- D) To specify the number of member functions in the class

Answer: A) As a constant value within the class

8. How can a friend class declared inside a template class access private data?

- A) Only the friend functions can access private data, not friend classes.
- B) Friend classes can access private data of all template instantiations.
- C) Friend classes can only access private data of the specific instantiation of the template class they are friends with.
- D) Friend classes have no special access to private data.

Answer: B) Friend classes can access private data of all template instantiations.

9. What is the role of the `template <class T>` declaration in the context of class templates?

- A) It defines the specific data type to be used in the class.
- B) It declares a placeholder for a data type to be used in the class.
- C) It declares the class as a template that can be instantiated with multiple data types.
- D) It specifies the non-type parameters of the class.

Answer: B) It declares a placeholder for a data type to be used in the class.

10. What is the impact of using templates on the compile-time performance of a program?

- A) Templates can slow down compilation due to code generation for different types.
- B) Templates have no impact on compile-time performance.
- C) Templates improve compile-time performance by reducing code complexity.
- D) Templates make compilation faster by optimizing the code for specific types.

Answer: A) Templates can slow down compilation due to code generation for different types.

. What is the primary purpose of the `Matrix` class described in the problem?

- A) To handle very large matrices
- B) To perform basic matrix operations such as addition, subtraction, and multiplication

- C) To handle string operations
- D) To manage memory allocation for arrays

Answer: B) To perform basic matrix operations such as addition, subtraction, and multiplication

2. What is the role of the constructor `Matrix(int rows, int cols)` in the `Matrix` class?

- A) To display the matrix
- B) To allocate memory for the matrix based on the number of rows and columns
- C) To perform matrix multiplication
- D) To initialize static members

Answer: B) To allocate memory for the matrix based on the number of rows and columns

3. How does the `Matrix` class handle matrix addition?

- A) By using a global function
- B) By overloading the + operator as a member function
- C) By using a static method
- D) By using a friend function

Answer: B) By overloading the + operator as a member function

4. Which operator overload function should be used to handle the addition of a scalar to a matrix where the scalar is on the left side?

- A) `Matrix operator + (const Scalar) const;`
- B) `friend Matrix operator + (const Scalar, const Matrix &);`
- C) `Matrix operator + (Matrix &) const;`
- D) `friend Matrix operator + (Matrix &, const Scalar);`

Answer: B) friend Matrix operator + (const Scalar, const Matrix &);

5. When implementing matrix subtraction, how is the operation `A - x` handled?

- A) By overloading the - operator as a friend function
- B) By overloading the - operator as a member function
- C) By using a global function
- D) By using a static method

Answer: B) By overloading the - operator as a member function

6. Which operator overload function is used to handle the multiplication of a

matrix by a scalar where the scalar is on the left side?

- A) Matrix operator * (const Scalar) const;
- B) friend Matrix operator * (const Scalar, const Matrix &);
- C) Matrix operator * (const Matrix &) const;
- D) friend Matrix operator * (const Matrix &, const Scalar);

Answer: B) friend Matrix operator * (const Scalar, const Matrix &);

7. For which type of matrix operation would you use the member function `Matrix & transpose(void)`?

- A) Matrix addition
- B) Matrix multiplication
- C) Matrix transpose
- D) Matrix subtraction

Answer: C) Matrix transpose

8. What does the function `Matrix operator * (const Matrix &)` perform?

- A) Matrix transpose
- B) Scalar multiplication
- C) Matrix multiplication
- D) Matrix addition

Answer: C) Matrix multiplication

9. How is matrix subtraction handled when a scalar is subtracted from a matrix (e.g., `x - A`)?

- A) By overloading the - operator as a member function
- B) By overloading the - operator as a friend function
- C) By using a global function
- D) By using a static method

Answer: B) By overloading the - operator as a friend function

10. What is the primary purpose of overloading the `<<` and `>>` operators for the `Matrix` class?

- A) To perform matrix multiplication
- B) To display and input matrix data
- C) To handle matrix transpose
- D) To allocate memory dynamically

Answer: B) To display and input matrix data

11. What does the Matrix operator / (const Scalar) function perform?

- A) Matrix division by a scalar
- B) Matrix addition with a scalar
- C) Scalar multiplication of a matrix
- D) Matrix subtraction by a scalar

Answer: A) Matrix division by a scalar

12. How does the Matrix class handle the operation A + x where x is a scalar?

- A) By overloading the + operator as a member function
- B) By overloading the + operator as a friend function
- C) By using a global function
- D) By using a static method

Answer: A) By overloading the + operator as a member function

13. What happens to the dimensions of a matrix when it is transposed?

- A) The number of rows becomes the number of columns and vice versa.
- B) The matrix is resized to have twice the original number of rows and columns.
- C) The number of rows and columns remains the same.
- D) The matrix becomes a square matrix.

Answer: A) The number of rows becomes the number of columns and vice versa.

14. What is the role of a friend function in the Matrix class?

- A) To access private members of the class
- B) To allocate memory for the matrix
- C) To handle matrix addition and subtraction
- D) To display matrix elements

Answer: A) To access private members of the class

15. Which operator overload function is used to handle the multiplication of two matrices?

- A) Matrix operator * (const Matrix &);
- B) friend Matrix operator * (const Scalar, const Matrix &);
- C) Matrix operator * (const Scalar) const;
- D) friend Matrix operator * (const Matrix &, const Scalar);

Answer: A) Matrix operator * (const Matrix &);

Lecture#43:

, What is the primary operation performed to add two matrices?

- A) Subtracting the elements of one matrix from another
- B) Multiplying corresponding elements of two matrices
- C) Adding corresponding elements of two matrices
- D) Dividing corresponding elements of two matrices

Answer: C) Adding corresponding elements of two matrices

What must be true for two matrices to be added or subtracted?

- A) They must have the same number of rows and columns
- B) Their columns must be equal
- C) Their rows must be equal
- D) Their orders must be different

Answer: A) They must have the same number of rows and columns

What does scalar multiplication of a matrix involve?

- A) Multiplying a matrix by another matrix
- B) Multiplying a scalar by each element of the matrix
- C) Adding a scalar to each element of the matrix
- D) Subtracting a scalar from each element of the matrix

Answer: B) Multiplying a scalar by each element of the matrix

What is the condition for two matrices to be multiplied?

- A) The number of rows in the first matrix must be equal to the number of columns in the second matrix
- B) The number of columns in the first matrix must be equal to the number of rows in the second matrix
- C) Both matrices must be square
- D) Both matrices must have the same dimensions

Answer: B) The number of columns in the first matrix must be equal to the number of rows in the second matrix

 What does matrix transposition involve?

- A) Adding two matrices
- B) Subtracting a scalar from each element of a matrix
- C) Interchanging the rows and columns of the matrix
- D) Multiplying each element of a matrix by a scalar

Answer: C) Interchanging the rows and columns of the matrix

 In the context of the Matrix class, what does the member function `operator+` handle?

- A) Addition of a matrix to a scalar
- B) Addition of two matrices
- C) Multiplication of a matrix by a scalar
- D) Transposition of a matrix

Answer: B) Addition of two matrices

 How is the `transpose` member function used in the Matrix class?

- A) To add two matrices
- B) To subtract one matrix from another
- C) To interchange rows and columns of the matrix
- D) To multiply a matrix by another matrix

Answer: C) To interchange rows and columns of the matrix

 What is the purpose of the destructor in the Matrix class?

- A) To allocate memory for the matrix
- B) To perform matrix operations
- C) To deallocate memory previously allocated for the matrix
- D) To display the matrix

Answer: C) To deallocate memory previously allocated for the matrix

 What is the result of dividing a matrix by a scalar?

- A) Each element of the matrix is multiplied by the scalar
- B) Each element of the matrix is divided by the scalar
- C) Each element of the matrix is added to the scalar
- D) Each element of the matrix is subtracted by the scalar

Answer: B) Each element of the matrix is divided by the scalar

 How is a scalar addition to a matrix different from matrix addition?

- A) Scalar addition affects only the elements of one matrix
- B) Scalar addition is done element-wise, whereas matrix addition is done across matrices
- C) Scalar addition requires the matrix to have the same dimensions as the scalar
- D) Scalar addition and matrix addition are identical operations

Answer: B) Scalar addition is done element-wise, whereas matrix addition is done across matrices

 What is the role of friend functions in the Matrix class?

- A) To perform operations within the class itself
- B) To access private members of the class
- C) To provide external access to certain operations involving the class
- D) To allocate memory for the class

Answer: C) To provide external access to certain operations involving the class

 What is the result of multiplying two matrices?

- A) A matrix with the same dimensions as the original matrices
- B) A matrix with the number of rows of the first matrix and the number of columns of the second matrix
- C) A matrix with dimensions equal to the sum of the dimensions of the original matrices
- D) A matrix with dimensions equal to the difference of the dimensions of the original matrices

Answer: B) A matrix with the number of rows of the first matrix and the number of columns of the second matrix

 What is the purpose of the `operator*` member function in the Matrix class?

- A) To add two matrices
- B) To subtract a scalar from a matrix
- C) To multiply two matrices
- D) To transpose a matrix

Answer: C) To multiply two matrices

 When implementing matrix operations, why should matrices be passed by reference rather than by value?

- A) Passing by reference is faster and avoids unnecessary copying
- B) Passing by reference allows for copying of the matrix
- C) Passing by reference reduces memory allocation

- D) Passing by reference allows for matrix operations to be performed in parallel

Answer: A) Passing by reference is faster and avoids unnecessary copying

What is required for the default assignment operator to work properly with dynamically allocated memory?

- A) It must perform a deep copy of the object data members
- B) It must perform a shallow copy of the object data members
- C) It must not perform any copying
- D) It must allocate new memory for the object

Answer: A) It must perform a deep copy of the object data members

What is the purpose of the copy constructor in the Matrix class?

- A) To initialize a new Matrix object based on an existing one
- B) To allocate memory for the matrix
- C) To deallocate memory for the matrix
- D) To perform matrix operations

Answer: A) To initialize a new Matrix object based on an existing one

What is the result of adding a scalar to a matrix?

- A) A new matrix with each element increased by the scalar
- B) A matrix with each element multiplied by the scalar
- C) A matrix with each element divided by the scalar
- D) A matrix with each element decreased by the scalar

Answer: A) A new matrix with each element increased by the scalar

Which operation is performed using the `operator-` member function in the Matrix class?

- A) Addition of two matrices
- B) Subtraction of one matrix from another
- C) Multiplication of a matrix by a scalar
- D) Division of a matrix by a scalar

Answer: B) Subtraction of one matrix from another

What must be checked before performing matrix addition or subtraction?

- A) The number of rows and columns must be the same for both matrices
- B) The matrices must be square
- C) The matrices must have different dimensions

- D) The matrices must have different data types

Answer: A) The number of rows and columns must be the same for both matrices

 In the Matrix class, what is the purpose of the `operator/` member function?

- A) To add two matrices
- B) To subtract a scalar from a matrix
- C) To divide each element of the matrix by a scalar
- D) To transpose the matrix

Answer: C) To divide each element of the matrix by a scalar

 Which operator is overloaded as a friend function to handle matrix operations involving a scalar on the left-hand side?

- A) `operator+`
- B) `operator-`
- C) `operator*`
- D) `operator/`

Answer: C) `operator*`

 What is the role of the `display` function in the Matrix class?

- A) To allocate memory for the matrix
- B) To deallocate memory for the matrix
- C) To display the elements of the matrix
- D) To perform matrix operations

Answer: C) To display the elements of the matrix

 What is a key feature of the Matrix class constructor?

- A) It initializes a matrix with default values
- B) It deallocates memory for the matrix
- C) It allocates memory for the matrix based on specified dimensions
- D) It performs matrix operations

Answer: C) It allocates memory for the matrix based on specified dimensions

 What does the `operator+=` function do in the Matrix class?

- A) Adds a scalar to a matrix
- B) Adds two matrices together
- C) Multiplies two matrices

- D) Subtracts one matrix from another

Answer: B) Adds two matrices together

 Which operator is used to multiply a matrix by a scalar?

- A) operator+
- B) operator-
- C) operator*
- D) operator/

Answer: C) operator*

 What should be done to handle operations where one operand is a scalar and the other is a matrix?

- A) Use a member function for operations involving a matrix as the left operand
- B) Use a friend function for operations involving a scalar as the left operand
- C) Use a default function for all operations
- D) Use separate classes for scalars and matrices

Answer: B) Use a friend function for operations involving a scalar as the left operand

 How is matrix multiplication performed according to the provided text?

- A) By adding corresponding elements of the two matrices
- B) By multiplying corresponding elements of the two matrices
- C) By taking the dot product of rows and columns of the matrices
- D) By dividing corresponding elements of the two matrices

Answer: C) By taking the dot product of rows and columns of the matrices

 What happens to the dimensions of a matrix after transposition?

- A) The dimensions remain the same
- B) The number of rows and columns are swapped
- C) The dimensions are doubled
- D) The dimensions are halved

Answer: B) The number of rows and columns are swapped

 Why is it important to write a custom copy constructor for the Matrix class?

- A) To avoid deep copying and improve performance
- B) To perform a deep copy of the matrix to handle dynamic memory correctly
- C) To perform shallow copying of the matrix

- D) To display the matrix elements

Answer: B) To perform a deep copy of the matrix to handle dynamic memory correctly

 Which operator is used to subtract a scalar from a matrix?

- A) operator+
- B) operator-
- C) operator*
- D) operator/

Answer: B) operator-

Lecture # 43:

. *What is the purpose of a constructor in a C++ class?*

A. To destroy the object when it's no longer needed

B. To initialize the object when it is created

C. To copy the object to another instance

D. To overload operators for the class

Answer: B. To initialize the object when it is created

2. Which operator is used for dynamic memory allocation in C++?

A. delete

B. malloc

C. new

D. free

Answer: C. new

3. What is the main reason for defining a destructor in a class?

A. To initialize data members

B. To allocate memory dynamically

C. To deallocate memory and cleanup

D. To perform arithmetic operations

Answer: C. To deallocate memory and cleanup

4. In the Matrix class, what does the elements member represent?

A. A one-dimensional array of doubles

B. A pointer to a matrix object

C. A two-dimensional array of doubles

D. A vector of matrices

Answer: C. A two-dimensional array of doubles

5. What does the copy constructor do in the Matrix class?

A. Initializes a new matrix with default values

B. Allocates memory for a new matrix

C. Creates a copy of an existing matrix

D. Deletes an existing matrix

Answer: C. Creates a copy of an existing matrix

6. Which function is responsible for deep copying in the Matrix class?

A. Default constructor

B. Copy constructor

C. Destructor

D. Assignment operator

Answer: B. Copy constructor

7. What will happen if you don't define a destructor in a class that uses dynamic memory?

A. The compiler will automatically handle the memory

B. The program will crash

C. The allocated memory will not be deallocated, causing a memory leak

D. The memory will be automatically deallocated

Answer: C. The allocated memory will not be deallocated, causing a memory leak

8. Which keyword is used to ensure that a member function does not modify any member variables?

A. static

B. virtual

C. const

D. public

Answer: C. const

9. What does the getRows() function do in the Matrix class?

A. Sets the number of rows

B. Returns the number of columns

C. Returns the number of rows

D. Initializes the matrix rows

Answer: C. Returns the number of rows

10. Which function is used to read matrix data from a file in the Matrix class?

A. input()

B. output()

C. input(ifstream &is)

D. output(ofstream &os)

Answer: C. input(ifstream &is)

11. What is the purpose of the transpose() function in the Matrix class?

A. To swap rows and columns of the matrix

B. To add two matrices

C. To multiply a matrix by a scalar

D. To initialize the matrix with zeros

Answer: A. To swap rows and columns of the matrix

12. What does the operator+ function do in the Matrix class?

A. Subtracts two matrices

B. Adds two matrices or a matrix and a scalar

C. Multiplies two matrices

D. Divides a matrix by a scalar

Answer: B. Adds two matrices or a matrix and a scalar

13. What is the purpose of the friend keyword in C++?

A. To allow functions to access private members of a class

B. To make a class a friend of another class

C. To define global functions

D. To restrict access to members of the class

Answer: A. To allow functions to access private members of a class

14. Which operator is used to handle matrix addition with a scalar value?

A. operator-

B. operator*

C. operator+

D. operator/

Answer: C. operator+

15. What will the expression m + d do if m is a Matrix and d is a double?

A. Subtract d from each element of m

B. Multiply each element of m by d

C. Add d to each element of m

D. Divide each element of m by d

Answer: C. Add d to each element of m

16. How is memory allocated for a 2D matrix in the constructor of the Matrix class?

A. Using a single new statement

B. Using malloc and free

C. Using new with nested loops

D. Using std::vector

Answer: C. Using new with nested loops

17. What will happen if you use delete[] on a pointer allocated with new?

A. It will free the allocated memory correctly

B. It will cause a memory leak

C. It will corrupt the memory

D. It will crash the program

Answer: A. It will free the allocated memory correctly

18. In the Matrix class, what does the operator function with a double parameter do?*

A. Multiplies each element of the matrix by the double value

B. Adds the double value to each element of the matrix

C. Subtracts the double value from each element of the matrix

D. Divides each element of the matrix by the double value

Answer: A. Multiplies each element of the matrix by the double value

19. Which of the following functions would you use to read a matrix from standard input?

A. input(ifstream &is)

B. output(ostream &os)

C. input()

D. output(ifstream &is)

Answer: C. input()

20. What is the primary role of the assignment operator in the Matrix class?

A. To create a new matrix from scratch

B. To handle deep copying of matrix data

C. To transpose the matrix

D. To initialize the matrix with default values

Answer: B. To handle deep copying of matrix data

21. When would you use a const reference as a function parameter in the Matrix class?

A. To modify the object passed as a parameter

B. To avoid copying and allow the function to access the object without modifying it

C. To create a new matrix object

D. To allocate memory for the object

Answer: B. To avoid copying and allow the function to access the object without modifying it

22. What is the purpose of the operator- function with a double parameter in the Matrix class?

A. Subtracts each element of the matrix by the double value

B. Adds the double value to each element of the matrix

C. Multiplies each element of the matrix by the double value

D. Divides each element of the matrix by the double value

Answer: A. Subtracts each element of the matrix by the double value

23. What will the expression `d * m` do if `d` is a double and `m` is a Matrix?

A. Multiply each element of `m` by `d`

B. Add `d` to each element of `m`

C. Subtract `d` from each element of `m`

D. Divide each element of `m` by `d`

Answer: A. Multiply each element of `m` by `d`

24. What does the `transpose()` function return in the Matrix class?

A. A new matrix that is the transpose of the original matrix

B. The original matrix unchanged

C. A boolean value indicating success

D. A pointer to the transposed matrix

Answer: A. A new matrix that is the transpose of the original matrix

25. How does the Matrix class handle file output?

A. Using the `input()` function

B. Using the `output()` function with `ofstream`

C. Using the `transpose()` function

D. Using the `operator<<`

Answer: B. Using the `output()` function with `ofstream`

26. Which operator would you use to multiply two matrices?

A. `operator+`

B. operator-

C. operator*

D. operator/

Answer: C. operator*

Lecture # 45:

. **What is the primary role of the assignment operator in the Matrix class?**

- A) To compare two matrices
- B) To initialize a matrix
- C) To assign the value of one matrix to another
- D) To add two matrices

Answer: C) To assign the value of one matrix to another

Why is the assignment operator declared to return a reference to a matrix?

- A) To allow assignment chaining like a = b = c
- B) To avoid memory leaks
- C) To ensure that the original matrix is not modified
- D) To improve the efficiency of matrix operations

Answer: A) To allow assignment chaining like a = b = c

What is checked before performing the assignment operation to avoid self-assignment issues?

- A) The values of the matrices
- B) The size of the matrices
- C) If the memory addresses of the two matrices are the same
- D) The type of matrices

Answer: C) If the memory addresses of the two matrices are the same

In the assignment operator, what happens if the sizes of the matrices being assigned are different?

- A) The original matrix is overwritten

- B) A new matrix is created for the left-hand side object
- C) An error is thrown
- D) The assignment is aborted

Answer: B) A new matrix is created for the left-hand side object

What does the addition operator (+) for matrices return?

- A) A reference to the original matrix
- B) A boolean value indicating success
- C) A new matrix resulting from the addition
- D) An integer representing the size of the resultant matrix

Answer: C) A new matrix resulting from the addition

How does the += operator function for matrices differ from the + operator?

- A) It creates a temporary matrix
- B) It returns a reference to the modified left-hand side matrix
- C) It performs matrix multiplication
- D) It does not require conformability checks

Answer: B) It returns a reference to the modified left-hand side matrix

What is the purpose of the `operator + (double d)` function in the Matrix class?

- A) To add a double value to each element of the matrix
- B) To multiply a double value with each element of the matrix
- C) To divide each element of the matrix by a double value
- D) To subtract a double value from each element of the matrix

Answer: A) To add a double value to each element of the matrix

What is the role of the friend function `operator + (double d, Matrix &m)`?

- A) To add a matrix to a double value
- B) To add a double value to a matrix
- C) To subtract a matrix from a double value
- D) To divide a matrix by a double value

Answer: B) To add a double value to a matrix

What is checked before performing matrix multiplication in the `operator*` function?

- A) The matrices must have the same number of rows
- B) The number of columns in the first matrix must match the number of rows in

the second matrix
- C) The matrices must be of the same size
- D) The matrices must be square

Answer: B) The number of columns in the first matrix must match the number of rows in the second matrix

 What does the operator* function return after multiplying two matrices?

- A) A new matrix with dimensions determined by the multiplication
- B) A boolean indicating success
- C) A reference to the first matrix
- D) A reference to the second matrix

Answer: A) A new matrix with dimensions determined by the multiplication

 What precaution is taken in the operator/ function when dividing a matrix by a double?

- A) It checks if the divisor is zero and returns the original matrix if it is
- B) It throws an exception
- C) It performs the division anyway
- D) It returns an error message

Answer: A) It checks if the divisor is zero and returns the original matrix if it is

 Which operator is used to insert a matrix into an output stream?

- A) >>
- B) <<
- C) *
- D) +

Answer: B) <<

 Which function is used to read matrix data from the input stream?

- A) m.output()
- B) m.input()
- C) m.read()
- D) m.write()

Answer: B) m.input()

 What does the operator >> (ifstream &is, Matrix &m) function do?

- A) It writes matrix data to a file

- B) It reads matrix data from a file
- C) It inputs data from the keyboard
- D) It outputs matrix data to the screen

Answer: B) It reads matrix data from a file

 What is the correct function to output matrix data to an output file stream?

- A) ostream & operator << (ofstream & os, Matrix & m)
- B) ofstream & operator >> (ofstream & os, Matrix & m)
- C) ostream & operator << (ostream & os, Matrix & m)
- D) ofstream & operator << (ofstream & os, Matrix & m)

Answer: D) ofstream & operator << (ofstream & os, Matrix & m)

 In the Matrix class, which function is used to perform element-by-element addition of two matrices?

- A) operator+=
- B) operator+
- C) operator-
- D) operator*

Answer: B) operator+

 What should be avoided when using the assignment operator in C++?

- A) Self-assignment
- B) Multiple assignments in one statement
- C) Returning a reference
- D) Using the const keyword

Answer: A) Self-assignment

 How does the operator + (double d) function handle matrix addition?

- A) It returns a new matrix with each element incremented by the double value
- B) It modifies the original matrix directly
- C) It returns a reference to the original matrix
- D) It performs subtraction instead of addition

Answer: A) It returns a new matrix with each element incremented by the double value

 Which function checks if two matrices are conformable for addition or subtraction?

- A) operator *

- B) operator /
- C) operator +
- D) operator +=

Answer: C) operator +

What is the main difference between a while loop and a do-while loop?

- A) A while loop always executes at least once, while a do-while loop does not.
- B) A do-while loop always executes at least once, while a while loop does not.
- C) Both loops execute the same number of times.
- D) There is no difference between them.

Answer: B) A do-while loop always executes at least once, while a while loop does not.

In the context of pointers, what is a dangling pointer?

- A) A pointer that points to a valid memory location
- B) A pointer that has been deallocated but still points to the old location
- C) A pointer that is correctly initialized
- D) A pointer that points to the beginning of an array

Answer: B) A pointer that has been deallocated but still points to the old location

What does the operator - function do in the Matrix class?

- A) It adds two matrices
- B) It multiplies two matrices
- C) It subtracts one matrix from another
- D) It divides one matrix by another

Answer: C) It subtracts one matrix from another

Which programming constructs are fundamental for solving any problem according to the text?

- A) Functions and classes
- B) Sequences, decisions, and loops
- C) Arrays and pointers
- D) Variables and operators

Answer: B) Sequences, decisions, and loops

What is a primary feature of modern languages like Java regarding memory management?

- A) Manual memory allocation and deallocation
- B) Automatic garbage collection
- C) No need for pointers
- D) Direct memory access

Answer: B) Automatic garbage collection

 ow is the function operator+ (double d, Matrix &m) defined?

- a) As a member function of the Matrix class
- b) As a friend function of the Matrix class
- c) As a static function of the Matrix class
- d) As a global function
- Answer: b) As a friend function of the Matrix class

 What does the operator* function do when multiplying two matrices?

- a) It adds the matrices element-wise
- b) It returns a matrix with dimensions equal to the number of rows of the first matrix and the number of columns of the second matrix
- c) It divides the matrices element-wise
- d) It returns the sum of all elements in the matrices
- Answer: b) It returns a matrix with dimensions equal to the number of rows of the first matrix and the number of columns of the second matrix

 What must be checked before performing matrix multiplication?

- a) The matrices must have the same number of rows and columns
- b) The number of rows of the first matrix must equal the number of columns of the second matrix
- c) The matrices must be square matrices
- d) The elements must be integers
- Answer: b) The number of rows of the first matrix must equal the number of columns of the second matrix

 How does the operator+ (double d) function handle the matrix when d is a double value?

- a) It returns a new matrix with d added to every element
- b) It modifies the original matrix by adding d to every element
- c) It multiplies every element of the matrix by d
- d) It subtracts d from every element of the matrix
- Answer: a) It returns a new matrix with d added to every element

 What happens if division by zero is attempted in matrix operations?

- a) The program will throw an exception

- b) The original matrix is returned without performing the division
- c) The program will crash
- d) The matrix elements are set to zero
- **Answer:** b) The original matrix is returned without performing the division

 What is the purpose of the stream insertion operator `operator<<` in the `Matrix` class?

- a) To read matrix data from the keyboard
- b) To write matrix data to an output stream
- c) To write matrix data to a file
- d) To read matrix data from a file
- **Answer:** b) To write matrix data to an output stream

 How is the stream extraction operator `operator>>` used for input?

- a) It reads matrix data from a file
- b) It reads matrix data from the keyboard
- c) It writes matrix data to the console
- d) It converts matrix data to a string
- **Answer:** b) It reads matrix data from the keyboard

 What is the significance of using `const` in the `operator=` declaration?

- a) To make sure the returned matrix cannot be modified
- b) To ensure the assignment operator does not alter the matrix being assigned from
- c) To prevent the matrix from being reallocated
- d) To make the matrix immutable
- **Answer:** a) To make sure the returned matrix cannot be modified

 What is the result of `a = b + c` where `a`, `b`, and `c` are matrices?

- a) `a` is assigned the sum of `b` and `c`
- b) `a` is unchanged
- c) `b` is assigned the sum of `a` and `c`
- d) `c` is assigned the sum of `a` and `b`
- **Answer:** a) `a` is assigned the sum of `b` and `c`

 Which operator function is used to multiply a matrix by a scalar value?

- a) `operator+`
- b) `operator-`
- c) `operator*`
- d) `operator/`
- **Answer:** c) `operator*`

 How does the operator* function handle matrix multiplication?

- a) It performs element-wise multiplication
- b) It calculates the dot product of rows and columns
- c) It adds the matrices element-wise
- d) It returns the product of matrix dimensions
- Answer: b) It calculates the dot product of rows and columns

 In the context of the Matrix class, what is the role of the delete [] elements; statement in the assignment operator?

- a) It reallocates memory for a new matrix
- b) It frees the memory of the current matrix
- c) It initializes the matrix elements
- d) It copies the matrix elements
- Answer: b) It frees the memory of the current matrix

 What does the Matrix temp(*this); statement do in the operator+ function?

- a) It copies the current matrix to a temporary matrix
- b) It creates a new matrix with the same size as the original
- c) It initializes a new matrix with zero values
- d) It performs a matrix subtraction
- Answer: a) It copies the current matrix to a temporary matrix